

Assignment 2 Design Rationale

Group E

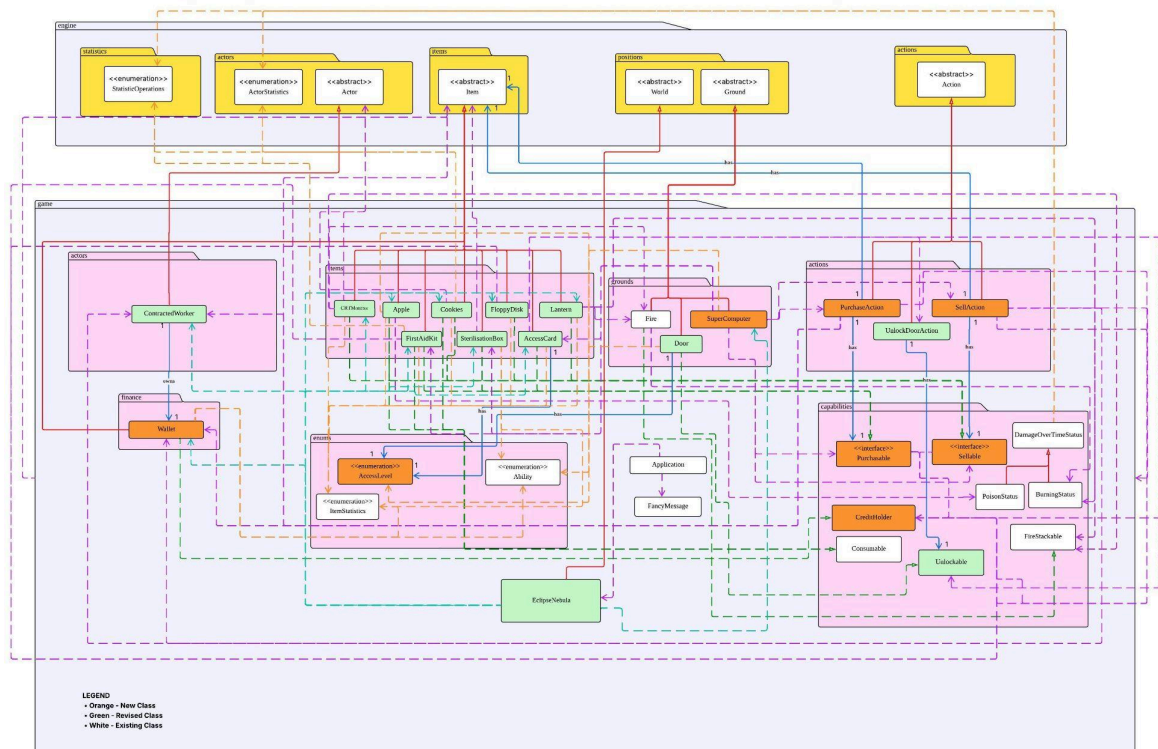
Suchir Ganesh Jaiganesh (35247096)

Chathya Attanayake (35427965)

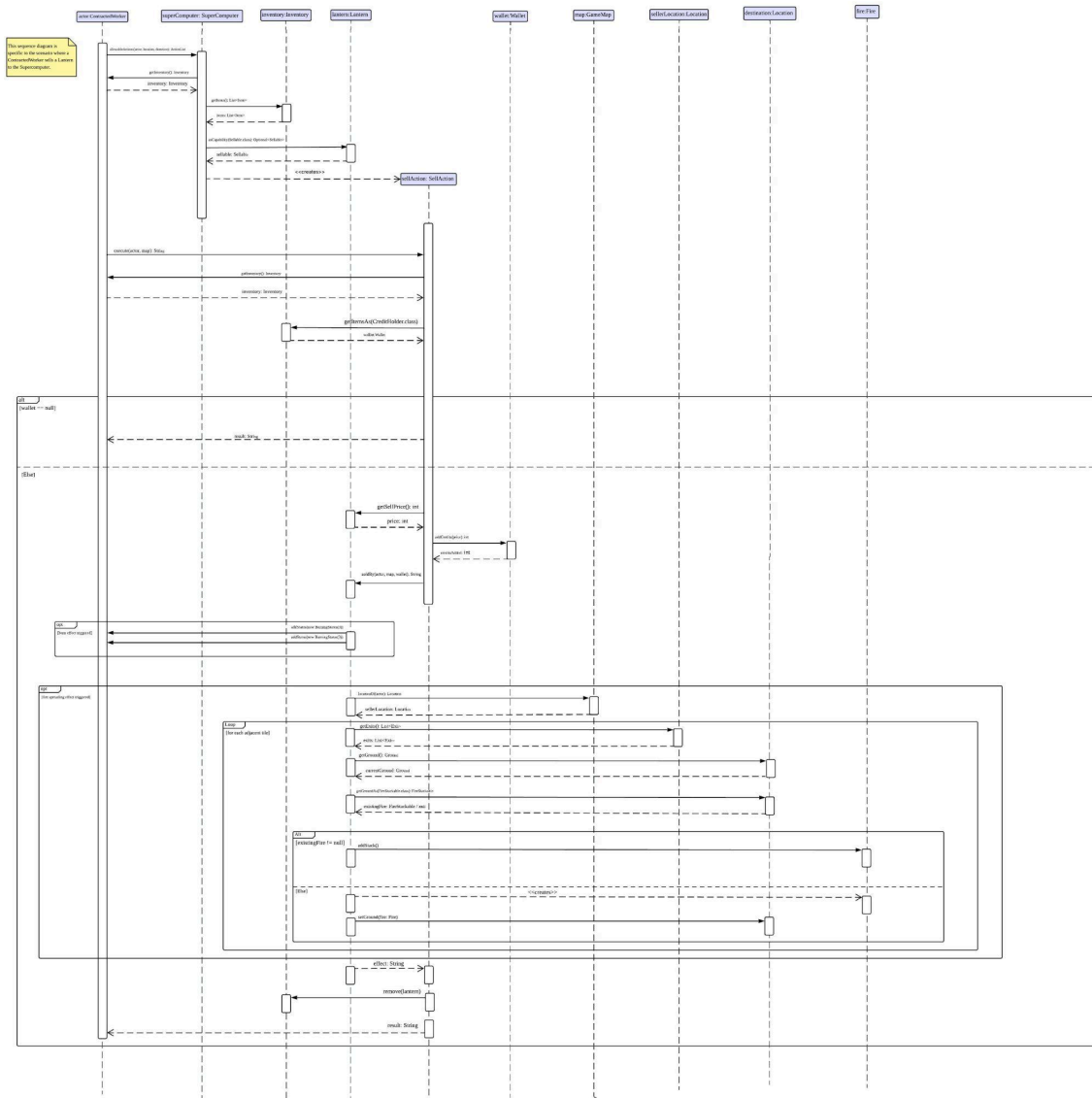
Jewell Eudocia Gomes (35627239)

Victoria Tay Wen Xie (35181257)

Aida Ab Aziz (34597506)



REQ 1 Sequence Diagrams



REQ1 Design Rationale

A. Supercomputer Transaction Handling

SellAction and PurchaseAction are used as reusable transaction actions, supported by Sellable and Purchasable interfaces that define economy contracts for items.

Design 1: Implement all buying and selling logic directly inside the SuperComputer class

Pros	Cons
Simple Implementation: All buying and selling logic is written in one place, making it easy to follow at first.	Violation of SRP: SuperComputer would be responsible for displaying options, checking wallet balance, updating inventory, modifying credits, and applying item-specific effects.
No Abstraction Overhead: Avoids creating extra interfaces and action classes.	Code Duplication: Similar transaction steps such as checking credits, adding credits, deducting credits, and removing items would be repeated for different items.
Direct Logic Flow: The effect of each transaction can be seen directly inside SuperComputer.	Low Extensibility: Adding a new economy item would require modifying SuperComputer, which violates OCP.
Fewer Initial Files: The design is faster to implement for a very small prototype.	High Coupling: SuperComputer would become dependent on concrete item classes such as Apple, Cookies, Lantern, FloppyDisk, and CRTMonitor.

Design 2: Implement generalised economy interfaces using Sellable, Purchasable, and reusable transaction actions.

Pros	Cons
Promotes Code Reuse (DRY): A single SellAction handles any item implementing Sellable, and a single PurchaseAction handles any item implementing Purchasable.	Increased Complexity: Requires extra interfaces and action classes compared to placing all logic inside SuperComputer.
High Extensibility (OCP): New economy items can be added by implementing Sellable or Purchasable without rewriting the existing action classes.	Indirect Logic: The final item effect is triggered through polymorphism, so the effect is not written directly inside the action class.

Decoupling (DIP): SellAction and PurchaseAction depend on abstractions rather than concrete item classes.	Slight Abstraction Overhead: For a very small economy system, interfaces may feel unnecessary at first.
Separation of Concerns (SRP): SuperComputer provides actions, action classes manage transaction flow, and item classes manage their own specific effects.	Indirect Execution: The final item effect is triggered through polymorphism, so the effect is not written directly inside the action class.
Interface Segregation (ISP): Only items that can be sold implement Sellable, and only items that can be bought implement Purchasable.	Higher Initial Structure: The design introduces extra interfaces and action classes compared to a direct implementation.

Justification: Design 2 was chosen because REQ1 contains many items with different prices and immediate transaction effects. Selling an Apple can poison the worker, selling Cookies deducts health based on remaining quantity, selling a Lantern can burn the worker or spawn fire, and selling a Floppy Disk can trigger a credit glitch. If all of these effects lived inside SuperComputer, the class would become too large and too dependent on the details of every item. With the final design, SuperComputer only checks which inventory items implement Sellable and adds the appropriate SellAction, it never needs to know what each item actually does on sale. That logic stays inside the item through soldBy(). SellAction handles the shared transaction steps, finding the wallet, adding credits, removing the item, then delegates the special effect to the item. This keeps SRP intact and means adding new economy items only requires implementing the right interface, not touching the existing action classes. The trade-off is that the logic is slightly less direct, but given how many items with different effects REQ1 requires, that indirection is justified.

B. Wallet and Credit Management

A Wallet item implementing the CreditHolder interface is used to encapsulate all credit storage and enforcement rules for the worker's economy system.

Design 1: Store credits directly as an integer field inside ContractedWorker

Pros	Cons
Simple Implementation: Credits can be stored and updated directly inside the worker class.	Violation of SRP: ContractedWorker would be responsible for both actor behaviour and credit management rules.
No Additional Object Required: No need to create a separate Wallet class or CreditHolder interface.	Weak Encapsulation: Other classes may directly modify the credit value without consistently enforcing the 1,000-credit limit.

Easy Access: Actions can directly call getter and setter methods on the worker.	High Coupling: Economy actions become dependent on ContractedWorker instead of a reusable credit abstraction.
Fewer Initial Classes: Faster to implement for a very small prototype.	Low Extensibility: If another actor needs to hold credits in the future, the same logic would need to be duplicated.
Direct Logic Flow: The worker's credits can be seen as part of the worker's state.	Poor Maintainability: Any change to credit rules such as the maximum limit would require modifying the worker or every place credits are changed.

Design 2: Create a Wallet item that implements the CreditHolder interface (Current Design)

Pros	Cons
Encapsulation: Credit storage, deduction, forceful deduction, and the 1,000-credit maximum are all managed inside Wallet.	Increased Complexity: Requires an additional item class and a credit interface.
Promotes Code Reuse (DRY): Any future object that needs to store credits can implement CreditHolder instead of duplicating wallet logic.	Indirect Access: Actions must search the worker's inventory to find an object implementing CreditHolder.
Decoupling (DIP): SellAction and PurchaseAction depend on the CreditHolder abstraction rather than directly on Wallet or ContractedWorker.	Slight Runtime Overhead: The inventory must be checked before each transaction.
Strong Requirement Alignment: The 1,000-credit limit is enforced in one place through addCredits(), preventing inconsistent credit updates.	The UML must clearly show that Wallet realises the CreditHolder interface.
Supports SRP: ContractedWorker focuses on actor behaviour while Wallet focuses on credit management.	Inventory Dependency: The wallet must remain available in the actor's inventory for transactions to work correctly.
Improves Maintainability: If the maximum credit capacity changes, only Wallet needs to be modified.	Requires careful implementation to ensure the wallet is not accidentally removed like a normal portable item.

Justification: Design 2 was chosen because the requirement states that each worker's wallet can hold a maximum of 1,000 credits. That rule needs one central place to live, if it were stored as a raw integer on the worker, nothing would stop other classes from bypassing it or applying it inconsistently. By placing the rule inside Wallet, the system has one clear location that controls how credits are added, deducted, and capped. SellAction and PurchaseAction

depend on CreditHolder rather than the concrete Wallet class, so they only care that the object can manage credits, not how it stores them internally. The trade-off is that actions must locate the wallet from the inventory rather than accessing a direct field, and the wallet had to be marked as non-portable and essential to prevent it from being accidentally dropped. But the benefit is that if the credit limit ever changes, only Wallet needs updating, none of the transaction classes are affected.

C. Access Card Level Design

One AccessCard class supported by an AccessLevel enum is used to represent all card levels without duplicating common card behaviour across separate subclasses.

Design 1: Make AccessCard abstract and create separate subclasses for each card level

Pros	Cons
Clear Class Representation: Each card level has its own class such as LevelOneAccessCard, LevelTwoAccessCard, and LevelThreeAccessCard.	Code Duplication (DRY Violation): Each subclass would repeat similar fields such as name, display character, purchase price, weight, and clearance rank.
Direct Specialisation: Card-specific purchase effects can be written inside each subclass.	Unnecessary Inheritance: The cards mainly differ by data values rather than completely different responsibilities.
Easy to Visualise in UML: The class hierarchy clearly shows that all card levels are access cards.	Maintenance Difficulty: Changing shared access-card behaviour may require editing several subclasses.
Useful for Very Different Behaviours: Subclasses would be appropriate if each card level had completely different algorithms.	Class Clutter: The items package would contain several small classes that mostly perform the same role.
Supports LSP if designed carefully: Each level card could still be substituted wherever an AccessCard is expected.	Weak KISS: The design becomes more complicated than necessary when the difference between cards is mostly fixed data.

Design 2: Use one AccessCard class supported by an AccessLevel enum (Current Design)

Pros	Cons
Promotes Code Reuse (DRY): One AccessCard class represents all access levels without repeating common card behaviour.	Conditional Logic: AccessCard may still need small checks for level-specific purchase effects.
Centralised Data: The AccessLevel enum stores rank, card name, display character, purchase price, and weight in one place.	Enum Dependency: AccessCard depends on AccessLevel for its fixed card properties.
Strong Requirement Alignment: The canOpen() method directly supports the rule that higher-level cards can open lower-level doors.	Less Suitable for Very Different Cards: If future card levels have completely different behaviour, subclasses may become more appropriate.
Improves Maintainability: Changing the Level 2 card's price or weight only requires editing the enum instead of multiple subclasses.	UML Clarity Required: The diagram must clearly show <<enumeration>> AccessLevel and its relationship with AccessCard.
Follows KISS: Avoids unnecessary inheritance when the difference between cards is mostly stored data.	Some behaviour becomes data-driven, so the design is less visually separated than one-class-per-card.
Supports OCP for card data: A new access level can be added as a new enum value rather than creating another mostly duplicated subclass.	The enum approach must be reconsidered if future cards gain large, unrelated behaviours.

Justification: Design 2 was chosen because the three access cards are almost identical in structure, they all have a name, display character, purchase price, weight, and clearance rank. Creating three subclasses just to hold different data would duplicate a lot of the same code and produce unnecessary class clutter. The final design uses one AccessCard class backed by AccessLevel, which stores the fixed information per level and provides the canOpen() method. This directly supports the tiered security requirement because a Level 2 card can open Level 1 and Level 2 doors, while a Level 3 card can open all current doors, without duplicating the comparison logic anywhere. The main trade-off is that purchasedBy() contains small level-specific checks, such as Level 2 dealing 5 damage and Level 3 having a 50% hidden fee. However, these checks are limited, readable, and directly related to the selected AccessLevel, so they do not create the same duplication as three separate subclasses. If future cards became much more complex, refactoring into subclasses would be straightforward.

D. Door Unlocking and Clearance Checking

An Unlockable interface is used so that UnlockDoorAction depends on an abstraction rather than any concrete door class, allowing different door types to define their own access level and unlock effect.

Design 1: Let access cards or actions directly unlock concrete Door objects

Pros	Cons
Simple Implementation: The unlocking logic can directly target the Door class.	Tight Coupling: The access-card system becomes dependent on one concrete door implementation.
Easy to Understand: The card directly interacts with a door object.	Low Extensibility: Adding different door types would require more conditional checks or duplicated unlocking logic.
No Interface Overhead: Avoids creating an additional Unlockable interface.	Violation of DIP: High-level unlocking actions depend on concrete door classes instead of an abstraction.
Suitable for One Door Type: Works acceptably if the game only has one simple door.	Weak OCP: New locked objects would require changes to existing unlocking logic.
Direct Logic Flow: The unlock behaviour can be seen in one place.	Lower Cohesion: The action may become responsible not only for unlocking but also for knowing each door's special effect.

Design 2: Use an Unlockable interface for all locked environmental objects (Current Design)

Pros	Cons
Promotes Polymorphism: Any ground object can be unlocked if it fulfils the Unlockable contract.	Increased Complexity: Requires an additional interface and clearer UML relationships.
High Extensibility (OCP): New door types can be added by implementing Unlockable without modifying UnlockDoorAction.	Indirect Effect Flow: The action triggers the unlock effect through the Unlockable contract rather than containing the effect directly.
Decoupling (DIP): UnlockDoorAction depends on the Unlockable abstraction rather than concrete classes like AluminiumDoor or IronDoor.	Slight Abstraction Overhead: The interface may feel unnecessary if only one door type exists.

Supports SRP: UnlockDoorAction handles the unlocking process while each door handles its own access level and unlock effect.	The UML must clearly show the <<interface>> Unlockable stereotype.
Better Requirement Alignment: Different doors can have different effects such as shocking, spawning fire, or healing, while still using the same unlocking action.	More files and relationships must be maintained compared to a single concrete door solution.
Improves Maintainability: Door effects can change inside the relevant door class without modifying the action class.	Requires careful design to keep the diagram and implementation consistent.

Justification: Design 2 was chosen because the access-card design needs to support the tiered door system introduced from this point onwards, where different doors have different clearance requirements and unlock effects. If UnlockDoorAction depended on a concrete Door class, it would need conditional logic for each type, which breaks OCP. Using Unlockable means each door defines its own required access level and applyUnlockEffect(), the action just calls those methods without caring what type of door it is. AccessCard.allowableActions() also uses Unlockable to check adjacent tiles, comparing the card's level against the door's required level using canOpen(). This keeps clearance logic in one place on the enum rather than scattered across the card or action classes. The trade-off is another interface to maintain in the UML, but this is justified because it reduces coupling and lets the same unlocking process work across all current and future door types without modification.

Final Design Summary

The diagram above shows the final design of REQ1, which uses interface-based polymorphism to support both the economy system and the access-card system. Although this design introduces additional interfaces and action classes, it follows KISS by keeping each class focused on one clear responsibility instead of placing all buying, selling, wallet, and unlocking logic inside one large class.

The implementation uses Sellable and Purchasable as economy contracts. Items manage their own transaction effects through soldBy() and purchasedBy(), while SellAction and PurchaseAction focus on the reusable transaction process. This keeps special effects close to the items themselves, improving cohesion and reducing unnecessary coupling between the terminal and the items it trades.

The action classes also return descriptive transaction messages instead of relying on System.out.println, so the player can understand why credits, health, inventory, or status effects changed after buying, selling, or unlocking. This improves player feedback while keeping output behaviour aligned with the engine's action-return system.

The Wallet class implements CreditHolder to encapsulate all credit management rules. This ensures the 1,000-credit maximum is enforced in one place and prevents invalid credit values from being produced by external classes. If the wallet limit changes in the future, only

Wallet needs to be modified, ContractedWorker, SellAction, and PurchaseAction remain unchanged. This supports maintainability and the Single Point of Truth principle.

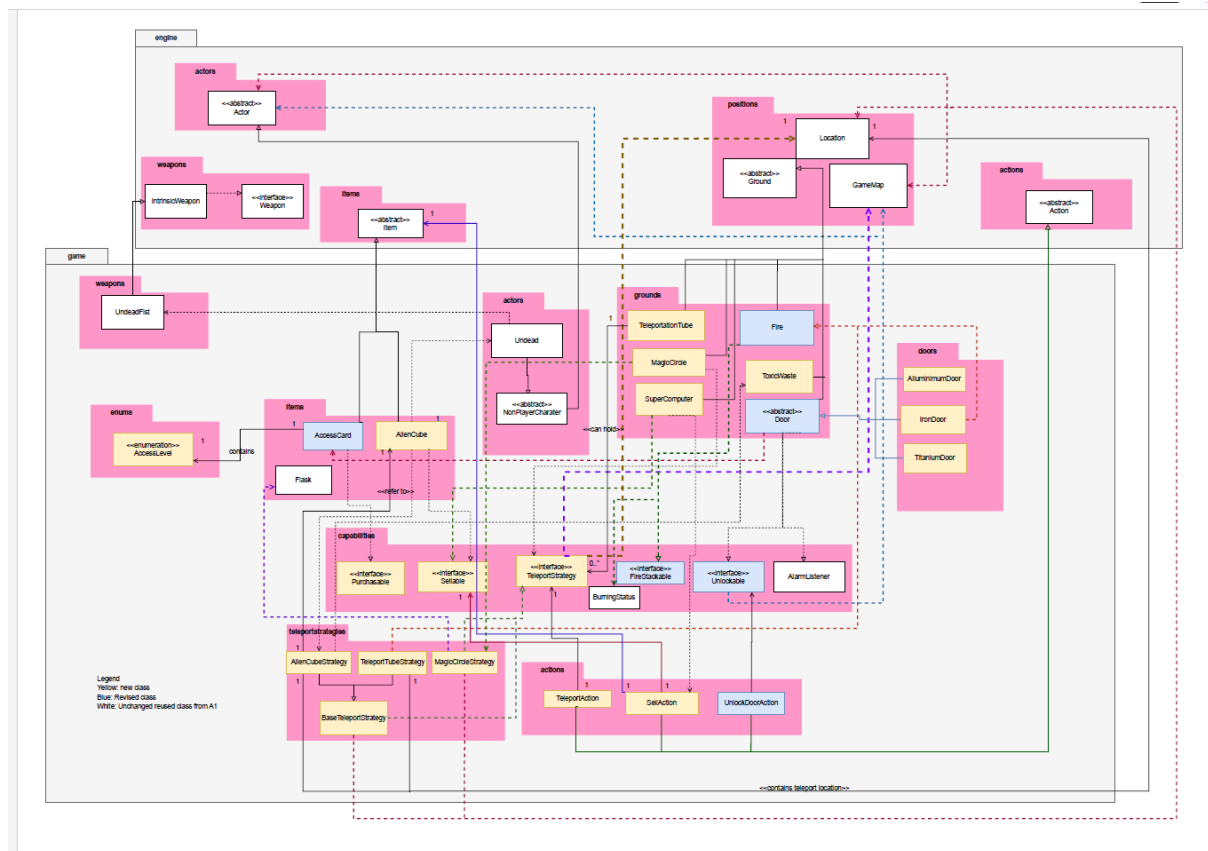
The access-card system uses an AccessLevel enum instead of separate subclasses for each card level. This decision was made because the cards mainly differ by fixed data such as rank, price, weight, name, and display character. The canOpen() method provides one consistent place for clearance comparison, ensuring higher-level cards can open lower-level doors without duplicating this logic.

The unlocking system uses the Unlockable interface so that different doors can define their own access requirement and unlock effect independently. This follows DIP because the unlocking action depends on an abstraction, and OCP because future locked objects can be added without changing the existing action.

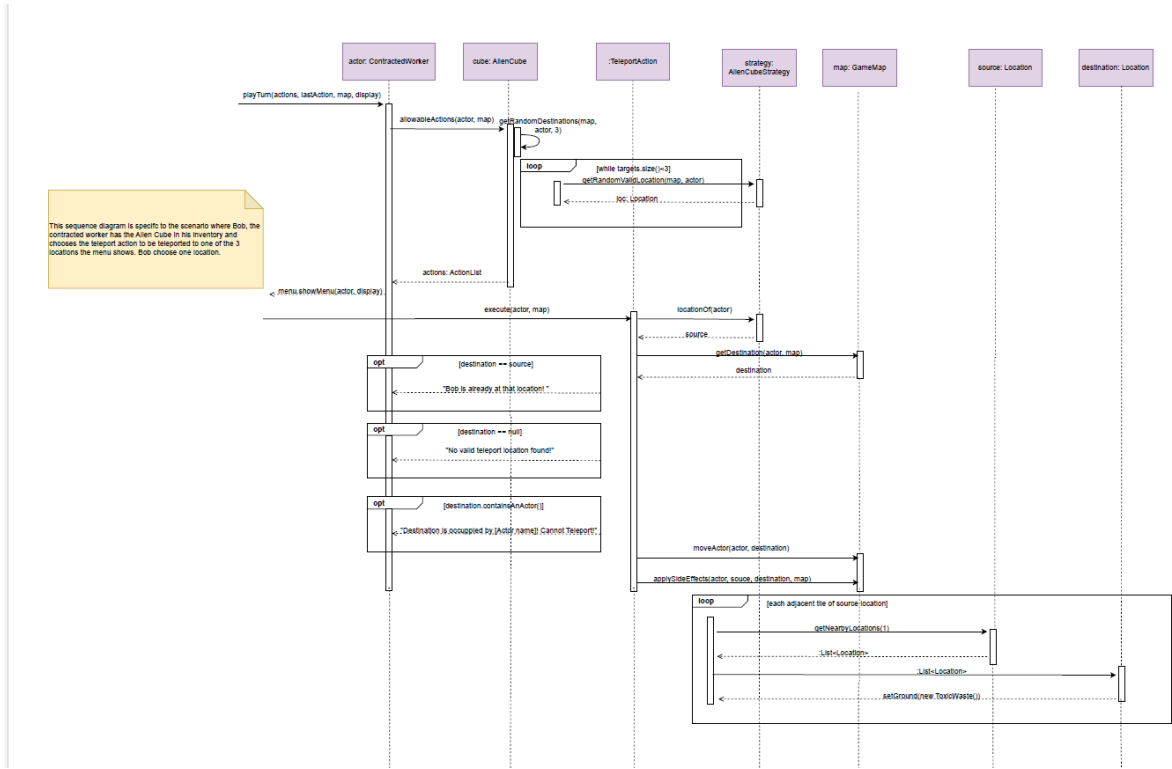
Hypothetical Extension Example: If a future requirement introduced a new sellable item such as an Alien Cube, the developer would only need to implement Sellable and define its selling price and effect. SellAction and SuperComputer would not require any changes. Similarly, if a new access card level was introduced, its rank, price, weight, and display character could be added to AccessLevel as a new enum value without creating another mostly duplicated subclass.

Trade-off Analysis: The primary trade-off in the chosen design is upfront abstraction overhead versus long-term maintainability. The cost includes creating additional interfaces such as Sellable, Purchasable, CreditHolder, and Unlockable, and ensuring the UML diagram clearly shows these relationships. Without these abstractions, transaction and unlocking logic would become duplicated across many concrete item and door classes. Since REQ1 contains multiple item effects, wallet rules, and access levels, the abstraction overhead is justified by reduced coupling, stronger cohesion, better extensibility, and clearer alignment with DRY, SRP, OCP, DIP, ISP, KISS, encapsulation, and polymorphism.

REQ 2 UML Class Diagram



REQ 2 Sequence Diagram



REQ2 Design Rationale

A. Teleportation Strategy Pattern

Design 1: Create separate action classes for each teleportation ground and item

Pros	Cons
Simple to understand for each teleportable	This violates DRY where duplicate teleport flow code, like the execute() method which has to be called multiple times for different teleport action classes made
Each separate action class for each teleportable follows the Single Responsibility Principle as each has unique behaviour	Adding new device requires new action class
	High connascence of algorithm as even small changes will need multiple edits

Design 2: Single TeleportAction class and TeleportStrategy interface.

Pros	Cons
DRY principle as teleport flow is only written once which is execute() and is reused for all teleportables	Requires understanding the strategy for each teleportable pattern
Open/Closed Principle as new device only require new strategy for that particular teleportable, no action modification needed	It will be more complex as it needs to be fitting to be used by the all teleportables
Dependency Inversion as action depends on abstraction, not concrete class	

Decision: Design 2 was chosen because the teleportation flow which is to get destination then check if it is a valid destination then it will move the worker then only apply side effects. These steps are identical for all devices. The few differences are destination selection and side effects for each teleportable. The strategy pattern encapsulates these variations reusing common flow, achieving DRY and Open/Closed Principle. For example, adding a new teleportable device requires only implementing TeleportStrategy. The TeleportAction will remain unchanged. This demonstrates low connascence between the action and teleportable devices.

B. Implementation of random location generation

Design 1: Implement random location generation separately in each strategy.

Pros	Cons
Each strategy is independent	Violates DRY as the random location logic is reused for both teleportation tube and Alien Cube
There is no inheritance hierarchy	High connascence of algorithm, bug fixes need multiple edits
It is simple to understand as it will just belong to whichever strategy it is supposed to act on	

Design 2: Created a new abstract BaseTeleportStrategy class with common random location method

Pros	Cons
DRY principle as the random logic written once and is shared by all other strategies that need it	A new abstract class has to created
Common algorithm in base, variations in subclasses	Tighter coupling through inheritance
Easier maintenance for shared teleport behaviour of acquiring random valid teleport destination	

Decision: I chose Design 2 as Teleportation Tube and Alien Cube need random location generation for their teleport logic. Centralising this idea in BaseTeleportStrategy follows DRY principle so that even with a new teleport device added that also acquired random teleport location, it can just inherit from the BaseTeleportStrategy.

C. Implementation of the teleport devices

Design 1: Handling malfunction and also the side effect in their teleport devices class

Pros	Cons
The logic is centralized and focus if all handle it themselves	This violates single responsibility as the teleportable device is doing too many things.
Very simple to locate the code	High coupling as it becomes too dependent on multiple different responsibilities as it now not only handles ground behaviour, teleportation but also malfunction effects and each different side effects
No extra classes other than the teleportable item or grounds	

Design 2: Encapsulate the behaviour in Strategy classes for each teleportable devices

Pros	Cons
Single Responsibility as the teleportation device remain either ground or item and then the strategy will handle teleport logic	One extra class for each teleportable device
KISS as each class does one thing and so easy to test	More interactions to trace during debugging
The teleportation device has low connascence with its teleport behaviour as they can be changed independently.	
Support Open/Closed Principle for adding behaviours for existing teleportable devices.	

Decision: I chose Design 2 as each teleportable device has unique requirements for their side effects and also some also react differently if it malfunctions. Encapsulating these in a separate strategy keeps the ground class simple and follows the Single Responsibility Principle where the teleportable device like teleportation tube do not need to know how teleportation works but it just calls the necessary class to implement it. For example, when a worker stands on teleport tube, allowableActions() will create one TeleportAction per strategy in its list. Each strategy know the destination and behaviour which shows that this is a low connascence design and the tube ground class should never changes even when new behaviours are added.

D. Design for the doors that now utilize a tiered security system

Design 1: Single Door class with clearance level and switch statement for effects

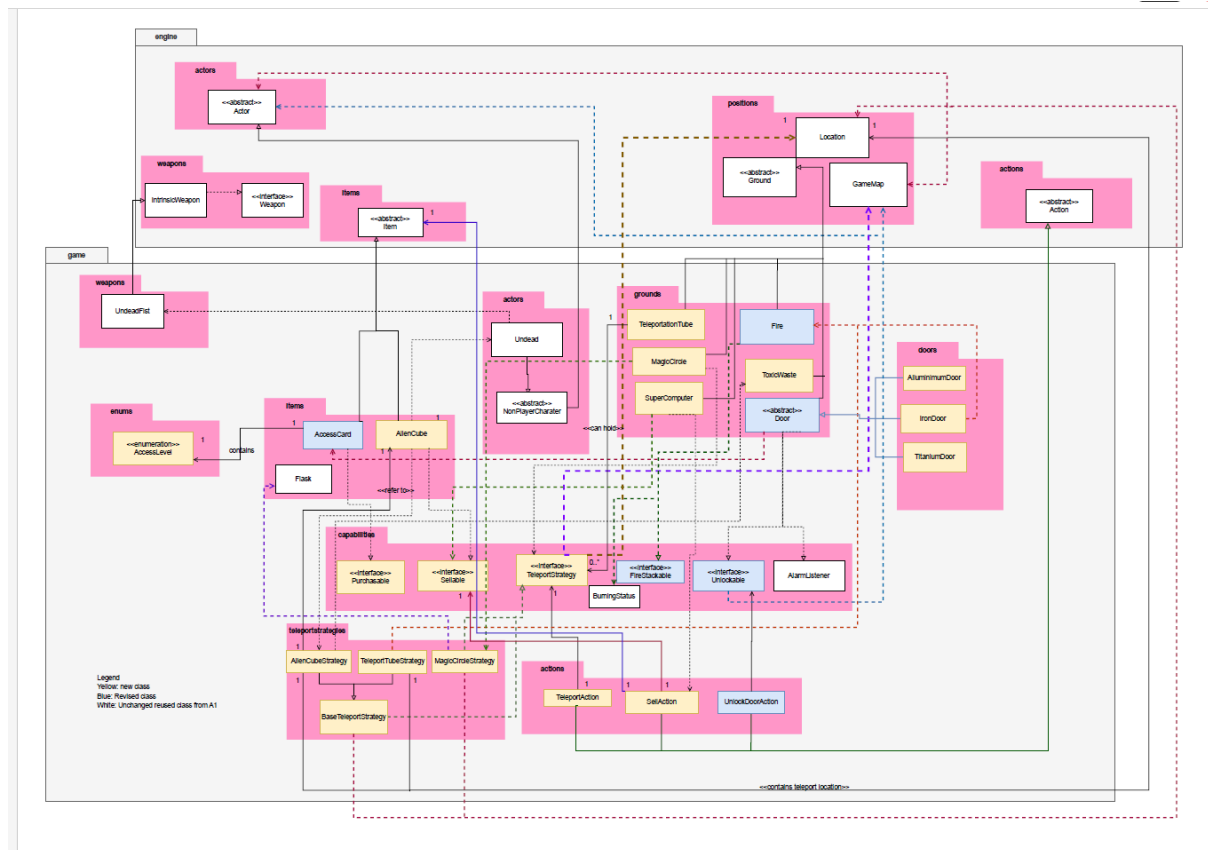
Pros	Cons
No need 3 different classes like AluminiumDoor, IronDor and TitaniumDoor classes but only Door class	Violates Open/Closed Principle as every time a new door type or clearance behaviour is introduced, then the existing switch or conditional logic inside the Door class must be modified
It is a very centralized logic so developers can know exactly where to look for door-related behaviour	High connascence as all security behavuiys are tightly connected inside one class.
The teleportation device has low connascence with its teleport behaviour as they can be changed independently.	No Liskov substitution principle as behaviours are handled through conditionals rather than specialized subclasses so it does not suppor substitute different door behaviours polymorphically.

Design 2: Extending from Door class for the new type of doors introduced that implemented the Unlockable interface

Pros	Cons
Open/Closed Principle as new door types added do not need to modify existing code	More classes needed as each door type now requires its own class implementation
It supports Liskov Substitution Principle as any door can replace another where Unlockable is expected	
Single Responsibility Principle as each door class handles its own effect.	

Decision: I chose Design 2 as each door has different effect and implementing them in different classes will support open/closed principle as the system is open for new door types but closed for modification with the use of Door being abstract class. For example, adding a new "Platinum Door" might requires only a new class that extends the Door abstract class that implements the Unlockable interface, no existing code has to be changed. Liskov Substitution principle can be seen where the system ensures that any door can be used whenever unlockable is expected.

Conclusion:



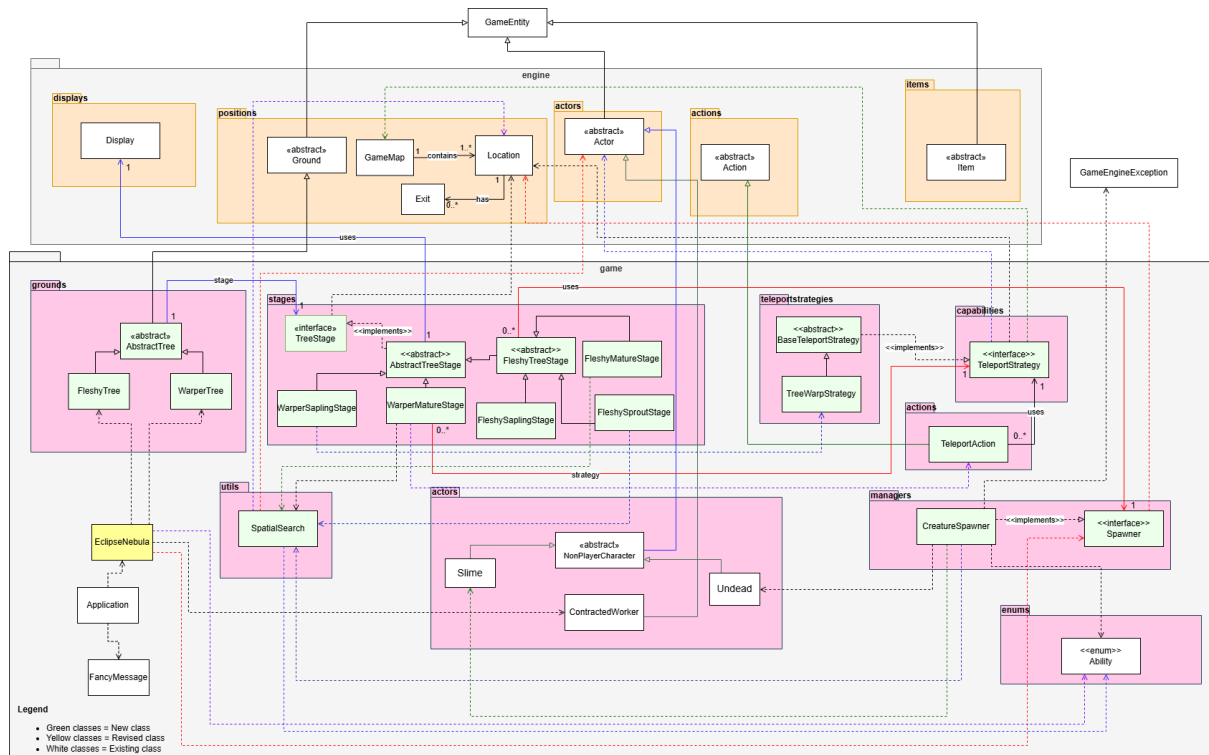
The new `TeleportAction` class extends the abstract `Action` class, while `BaseTeleportStrategy` and `MagicCircleStrategy` implement the `TeleportStrategy` interface. All teleportation devices inherit common execution flow from `TeleportAction` while encapsulating device-specific behaviour in their respective strategies. We abstract these identities to avoid repetitions in code (DRY), as all teleportation devices share the same core flow but differ in destination selection and side effects. `TeleportAction` has an attribute of type `TeleportStrategy` interface, which associates the action with the strategy. The interface acts as a level of abstraction between `TeleportAction` and concrete strategy classes. Removing dependencies between the concrete classes without changing the flow of control by diverting the dependency of both concrete classes to depend on the `TeleportStrategy` interface which is the Dependency Inversion Principle. All devices would implement the `TeleportStrategy` interface, no modifications need to be made to `TeleportAction` to support new devices which supports the Open/Closed principle.

The `Unlocked` interface defines the contract for all lockable objects: `unlock()`, `isUnlocked()`, `getRequiredAccessLevel()`, and `applyUnlockEffect()`. The abstract `Door` class implements this interface, providing common door state management. `AluminiumDoor`, `IronDoor`, and `TitaniumDoor` extend `Door` and each implement their own `applyUnlockEffect()` method, defining door-specific side effects that apply differently for different doors. `UnlockDoorAction` depends only on the `Unlocked` interface, not on specific door types which removes dependencies between the action and concrete doors which is the Dependency Inversion

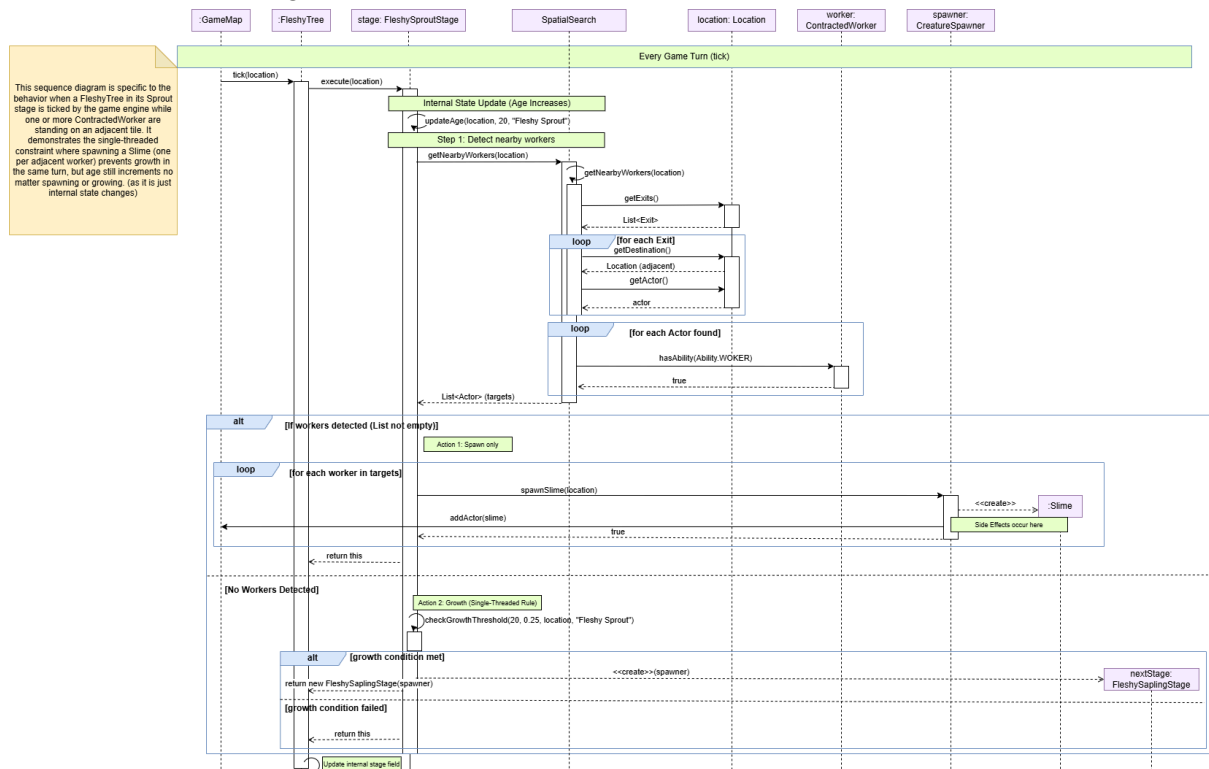
Principle. It also is designed in a way that allows extension where new door types can be added in future development which means that it follows Open/Closed Principle.

The REQ2 implementation demonstrates exemplary adherence to SOLID principles, low connascece, and DRY/KISS through the strategy pattern for teleportation devices and also abstractions based extension of door types. Each class has a single, clear responsibility, and dependencies are on abstractions rather than concrete implementations. The design is highly maintainable and extensible without modifying existing code. The separation of concerns through interfaces and strategies means that this system has low connascece.

REQ 3 UML Class Diagram



REQ 3 Sequence Diagram



Design Rationale

REQ 3

A. Implementation of Mutated Trees

New AbstractTree class that extends the engine's Ground class to serve as a base for FleshyTree and WarperTree.

Design 1: Directly implement tree logic in FleshyTree without AbstractTree.

Pros	Cons
KISS (Keep It Simple, Stupid): Easier to implement initially for a single moon with few objects. No need to manage a class hierarchy.	Violation of DRY Principle: Both trees share identical collision logic (canActorEnter) and lifecycle triggers (tick). Implementing these separately leads to duplicate code.
Fewer Files: Reduces the number of classes in the game.grounds package, making the file structure appear less cluttered.	High Maintenance Overhead: If the logic for how flora interacts with the environment changes (e.g., trees should now be flammable), the developer must update every tree class individually.

Design 2: Abstract Hierarchy (Current Design: AbstractTree as a base)

Pros	Cons
Code Reusability: Inheriting from Ground allows the trees to be placed on the GameMap and utilize the tick() method for per-turn logic.	Tighter coupling to the engine's Ground class, though necessary for environmental objects.
Simplified Stage Classes: Overriding canActorEnter() to return false centralizes the blocking behavior for all mutated flora stages (DRY).	Hypothetical Future Scenario: If a future requirement allows certain actors (like ghosts) to pass through trees, the logic would need to be modified.
Polymorphic Compatibility: Allows the engine to treat all stages as generic Ground while the tick() method ensures the tree's behavior evolves correctly.	
Adherence to LSP: Its stages never throw exceptions when called by the engine's tick() method, so we satisfy LSP. The engine can treat our complex tree hierarchy as a simple Ground object without knowing the difference.	

Justification: Design 2 was chosen to adhere to DRY and OCP. Both FleshyTree and WarperTree share the same ground behavior (blocking entry, delegating to stages). Abstracting this into a parent class prevents code duplication. The engine's Ground class is designed for extension, making this appropriate integration. The AbstractTree class extends the engine's Ground class. FleshyTree and WarperTree extend AbstractTree.

B. Tree Lifecycle and Growth Management

The design employs the State Pattern by creating a TreeStage interface and concrete stage classes (e.g., FleshySproutStage, WarperSaplingStage).

Design 1: Conditional Logic (if-else/switch) inside the Tree class

Pros	Cons
KISS: Simple to implement initially. All data is in one file.	Violation of OCP: Adding a new stage like Burnt Stage would require modifying the core Tree class, increasing the risk of bugs.
Fewer Files: Keeps all logic in one class, reducing the total number of files in the package.	Violation of SRP: The class becomes a God Class responsible for tracking age, handling spawning, managing growth chances, and rendering icons.
	Class Explosion: Significantly increases the number of classes in the system, which can make the project structure look more complex to new developers.
	Memory Overhead: Every tree on the map holds a reference to a stage object, which slightly increases memory usage compared to a simple primitive integer.

Design 2: State Pattern (Delegating to TreeStage objects) and Layered Abstraction with AbstractTreeStage

Pros	Cons
Adherence to SRP: Each stage (e.g., FleshySaplingStage) is only responsible for its own specific behavior and maturation logic.	Class Explosion: Significantly increases the number of classes in the system, which can make the project structure look more complex to new developers.
High Extensibility (OCP): New stages can be added by creating a new class without touching existing tree logic.	Memory Overhead: Every tree on the map holds a reference to a stage object, which slightly increases memory usage compared to a simple primitive integer.
Dynamic Behavior: The getDisplayChar() can change dynamically based on the current state, ensuring the map reflects the tree's growth correctly.	Inheritance Complexity: With AbstractTreeStage, it will add another layer to the stage hierarchy.
DRY Principle: Maturation math and internal aging are centralized in AbstractTreeStage. Concrete stages focus only on unique behavior.	
If a future requirement introduces a Burnt Tree Stage that spawns Fire elementals, the State Pattern allows adding FleshyBurntStage without modifying	

FleshyTree, FleshySproutStage, or FleshySaplingStage.	
---	--

Justification: Design 2 was chosen to satisfy SRP and OCP. The requirement specifies three distinct growth stages for Fleshy Trees and two for Warper Trees. The State Pattern encapsulates each stage's unique behavior (spawning, aging thresholds, growth chances) into separate classes. This eliminates if-else chains that would violate OCP. The connascence of control is reduced because the tree delegates to its current stage rather than determining behavior via type checking. The TreeStage interface defines the contract. FleshySproutStage, FleshySaplingStage, FleshyMatureStage, WarperSaplingStage, and WarperMatureStage implement this interface. The WarperMatureStage delegates the how of teleporting to a strategy, called the TreeWarpStrategy that adheres to the Open-Closed Principle. This ensures that the tree stage is decoupled from the specific mechanics of teleportation. The stage identifies the target, while the strategy handles the destination logic. This inversion of control facilitates high maintainability, as the warp behavior can be refactored or swapped without affecting the flora's lifecycle logic.

C. Separation of Spawning Concerns

Mature and Sprout stages must spawn Slimes or Undead. The design utilizes a Spawner interface and an intermediate FleshyTreeStage. The State Pattern reduces connascence of algorithms by encapsulating stage-specific behavior within separate classes. Without this pattern, the tree class would exhibit connascence of control, where a single method would need to know the logic for all growth stages via conditional statements. Using the Spawner interface reduces connascence of name between the flora system and creature spawning. The flora only needs to know the method name spawnSlime(), not the concrete class name CreatureSpawner.

Design 1: Trees directly instantiate Slime/Undead and handle environmental side effects.

Pros	Cons
KISS: Easier to implement for very small projects with only one or two stages.	Violation of OCP: Adding a new stage like Burnt Stage would require modifying the core Tree class, increasing the risk of bugs.
Lower Abstraction: Easier to read for a beginner as there is no need to jump between interfaces and concrete implementations.	Violation of SRP: The class becomes a God Class responsible for tracking age, handling spawning, managing growth chances, and rendering icons.
No Setup Needed: No need to pass a Spawner instance during the world initialization in EclipseNebula.	Class Explosion: Significantly increases the number of classes in the system, which can make the project structure look more complex to new developers.
	Memory Overhead: Every tree on the map holds a reference to a stage object, which slightly increases memory usage compared to a simple primitive integer.

	High Coupling: Trees become dependent on concrete creature classes. If the Slime constructor changes, the FleshySproutStage must also be updated.
--	---

Design 2: Trees depending on a Spawner interface and the FleshyTreeStage interface.

Pros	Cons
Dependency Inversion Principle (DIP): Trees depend on the Spawner abstraction. This keeps the flora system entirely separate from the creature spawning logic defined in Requirement 4. This inverts the dependency, ensuring that high-level flora logic is not coupled to low-level creature spawning implementation details. If a new spawner type (e.g., TestSpawner for unit tests) is created, the tree code remains unchanged.	Complex Setup: Requires dependency injection during map construction. EclipseNebula class needs to manage a global Spawner instance and inject it into the FleshyTree during map construction.
Interface Segregation Principle (ISP): By using an intermediate FleshyTreeStage, the WarperTree is not forced to hold a reference to a Spawner it doesn't need.	Dependency Tracking: Since the stage transitions happen at runtime, we must ensure the Spawner reference is correctly passed down to the next growth stage.
Maintainability: We can update how a Slime is spawned (e.g., making it drop items) in one place (CreatureSpawner) without touching a single line of tree code.	
Hypothetical Future Scenario: If Requirement 6 introduces 'Mutated Ferns' that spawn Spores, the existing Spawner interface can be extended with spawnSpore(), or a new FernSpawner can implement the same interface. The FleshyTree would not require changes.	
Adhered to connascence of name: The flora system depends only on the method name spawnSlime(), not the concrete CreatureSpawner class. If the implementation changes, like new side effects added, the flora system remains unaffected.	

Design 3: Global Spawner Dependency (All stages have a Spawner field)

Pros	Cons
Simplicity: All stages have access to the same tools regardless of tree type.	Violation of ISP: WarperTree stages (which only teleport) would be forced to carry a

	Spawner reference they never use.
--	-----------------------------------

Justification: Design 2 was chosen to satisfy DIP and ISP. The flora system should not depend on low-level creature spawning details defined in REQ4. By depending on the Spawner interface, the tree code remains unchanged if a new spawner implementation (e.g., TestSpawner for unit tests) is created. The intermediate FleshyTreeStage abstract class holds the Spawner reference, ensuring WarperTree (which does not spawn creatures) is not forced to carry an unused dependency. The Spawner interface declares spawnSlime(), spawnUndead(). CreatureSpawner implements this interface. FleshyTreeStage holds a reference to Spawner, while WarperTreeStage does not (ISP). By minimizing the connascence of name, the system achieves low coupling as each component knows as little as possible about others. This supports the OCP and makes the codebase resilient to requirement changes.

D. Spatial Search and Worker Detection

The design employs a static utility class SpatialSearch to detect nearby Ability.WORKER actors.

Design 1: Each stage implements its own exit traversal

Pros	Cons
KISS: Avoids the creation of an additional utility class. The logic is contained locally within the execute() method of each stage.	Violation of DRY: The 8 way exit traversal and Ability.WORKER check would be duplicated across at least four stage classes. Low Cohesion: Stage classes become responsible for low-level map navigation algorithms instead of purely focusing on growth logic.
Violate connascence of algorithms: Changing adjacent from 8 directional to 4 directional would require editing four files. Now, one change propagates everywhere.	Violation of SRP (Single Responsibility Principle): The primary role of a TreeStage should be managing growth transitions and stage specific effects. Burdening these classes with low-level spatial math reduces their functional cohesion.

Design 2: Centralized SpatialSearch utility (Current Design)

Pros	Cons
Eliminates Code Duplication (DRY): Prevents the need to write the same 8 way Exit traversal and capability check in every stage class (Sprout, Mature, etc.).	Static Coupling: Using static methods makes the stages tightly coupled to the SpatialSearch class, which can make unit testing more difficult compared to using an injected strategy.
Easier Maintenance: If the facility adds a new type of worker (e.g., a Security Guard with the same capability), the detection	Global Logic: If the search logic needs to be different for one specific tree (e.g., a tree that only reacts to players), the centralized

logic works automatically without updates.	utility might become cluttered with if statements.
Consistency: Ensures that both flora (REQ3) and environmental spawners (REQ4) use the exact same logic to find targets.	
Adhered to Connascence of Algorithms, in which the change of logic like the <code>getNearbyWorker()</code> will not need modifications in every class.	

Justification: Design 2 was chosen based on Ed discussion clarification. Without a centralized utility, the 8 way exit traversal would be duplicated across `FleshySproutStage`, `FleshyMatureStage`, `WarperMatureStage`, and `CreatureSpawner` (REQ4). This creates a connascence of algorithms. If detection logic changed, every class would need modification. Centralizing in `SpatialSearch.getNearbyWorkers()` ensures REQ3 and REQ4 use identical logic. The static coupling trade off is acceptable as this is infrastructure level utility code with no side effects. By minimizing the connascence of algorithms, the system achieves low coupling as each component knows as little as possible about others. This supports the OCP and makes the codebase resilient to requirement changes.

A1 to A2 Evolution Summary:

This centralized utility is a direct evolution from A1, where redundant code and type-checking were identified as maintenance risks. This refactor ensures REQ 3 flora and REQ 4 spawners share a single source of truth.

E. Single-Threaded Action Implementation

To fulfill the requirement that flora can only perform one action per turn, the design uses a specific branching structure in the `execute()` method.

Design 1: Priority-based Branching

Pros	Cons
Requirement Strictness: In <code>FleshySproutStage</code> , if a spawn occurs, the method returns this immediately. This ensures that a tree cannot grow and spawn a slime in the same tick.	Stunted Growth: A tree that is constantly surrounded by workers may never have the chance to grow because the spawning logic always takes precedence and exits the method early.
Deterministic Behavior: Providing a clear priority (Spawning > Growing) makes the game behavior predictable for the player.	State Leakage: We must manually ensure that the maturation threshold check is placed after the spawning check to prevent double actions.

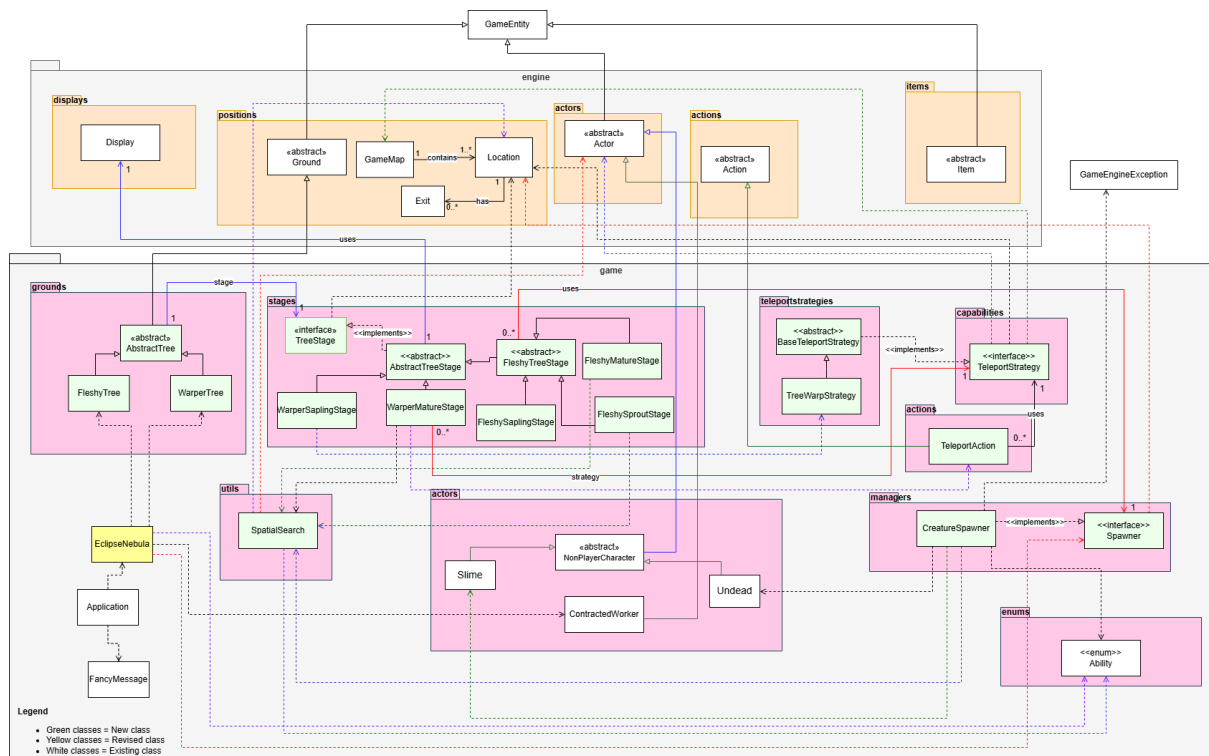
Design 2: Age Separation + Priority Branching

Pros	Cons
------	------

Fairness: The "Internal State Update" (aging) happens at the very start of the turn regardless of the outcome. This ensures the tree's clock is always ticking, even if its action was spent spawning a creature.

Subtle Logic: Requires the developer to understand the difference between an "internal update" and an "action," which might lead to confusion if the requirement is read too literally.

Justification: Design 2 was chosen based on Ed discussion clarification. The `updateAge()` method is called at the start of `execute()` regardless of spawning or growth, ensuring the tree's internal clock always ticks even when they are busy spawning Slimes. When workers are adjacent, `spawnSlime()` is called and the method returns this immediately, preventing the growth check from running in the same turn. This strictly satisfies the single threaded requirement while allowing aging to progress even when spawning occurs.



The diagram above shows the final design of Requirement 3, which utilizes Design 2 of parts A, B, C, D, and E stated above to implement a modular and extensible mutated flora system.

The new `AbstractTree` class extends the engine's abstract `Ground` class, while the concrete `FleshyTree` and `WarperTree` classes extend `AbstractTree`. Since all mutated trees share identical movement blocking behavior and the need to progress through growth stages, it is logical to abstract these common identities into a base class to avoid repetitions in code (DRY). The `AbstractTree` solely uses the engine's `tick()` method to delegate logic to a state object, ensuring that mutated trees can replace the usage of their parent `Ground` class without breaking the game loop's contract (Liskov Substitution Principle). By extending `Ground`, `AbstractTree` ensures it can be used anywhere the engine expects a `Ground` object without causing errors (LSP), as it respects the contract of the `tick` and `canActorEnter` methods.

The design implements the State Pattern via the `TreeStage` interface and its concrete implementations: `FleshySproutStage`, `FleshySaplingStage`, `FleshyMatureStage`,

WarperSaplingStage, and WarperMatureStage. Each class is exclusively responsible for the internal logic of a single stage, such as calculating maturation rolls or identifying workers for biological triggers. This satisfies the Single Responsibility Principle by ensuring that a change in the Sprout's behavior does not necessitate a modification of the Mature stage's code. Furthermore, this architecture follows the Open-Closed Principle, as new growth stages can be introduced by creating new classes without modifying the existing tree objects.

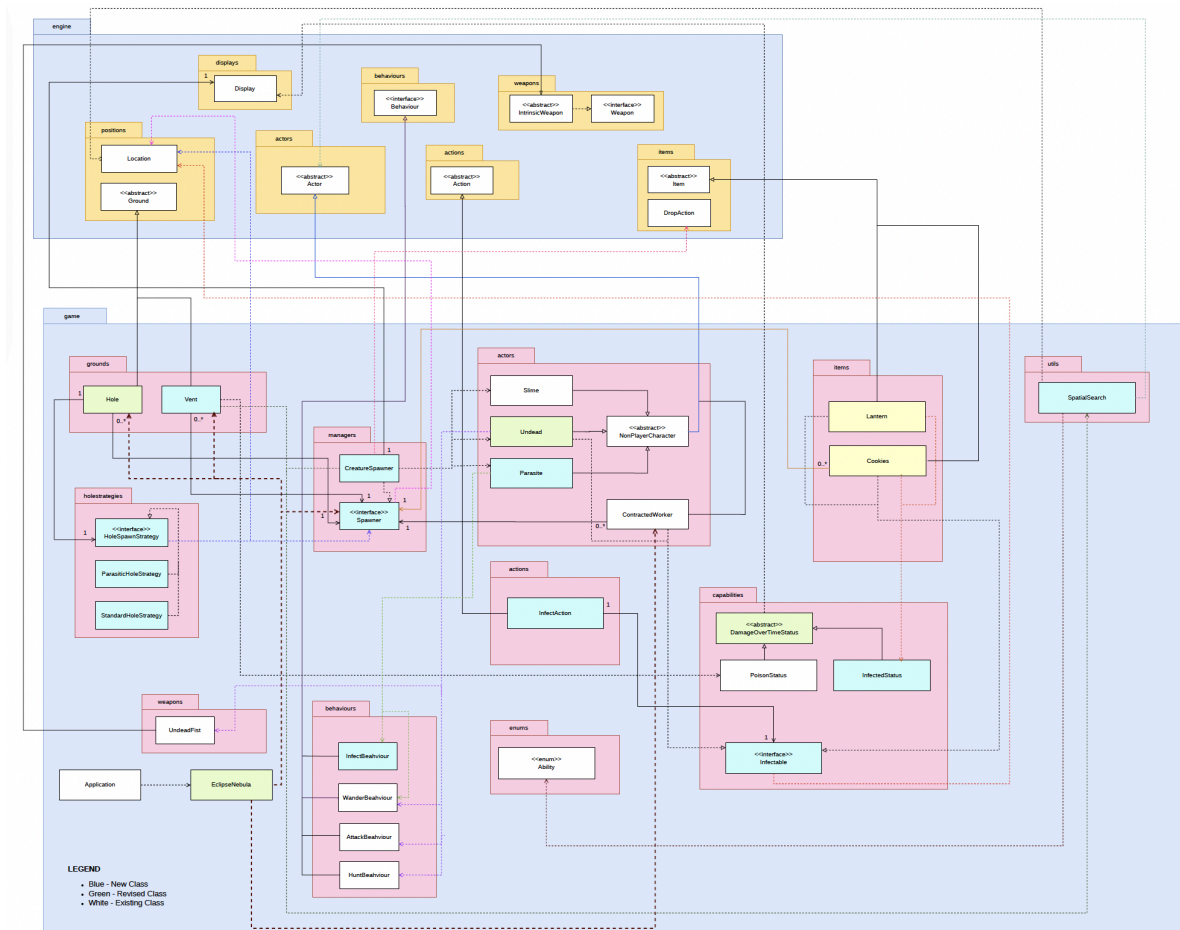
To integrate with the creature spawning mechanics of Requirement 4, the design utilizes the Spawner interface. Concrete stages of the Fleshy Tree depend on this abstraction rather than the concrete CreatureSpawner class. This ensures that the flora system remains unaware of the complex environmental reactions (such as slimes terrifying workers) that occur during spawning, successfully decoupling the two systems (Dependency Inversion Principle). By using the intermediate FleshyTreeStage abstract class to hold this dependency, we ensure that WarperTree stages are not burdened with a Spawner reference they do not require, thus adhering to the Interface Segregation Principle.

The implementation of the SpatialSearch utility class centralizes the logic for adjacent worker detection. By providing a common contract for map queries, we remove multiple dependencies of the stage classes on the engine's low-level Exit and Location iterators. This prevents the need to fix every individual stage class if the method of identifying targets changes, reducing the risk of code smell and promoting high Functional Cohesion across both the flora system and the creature spawning system (DRY).

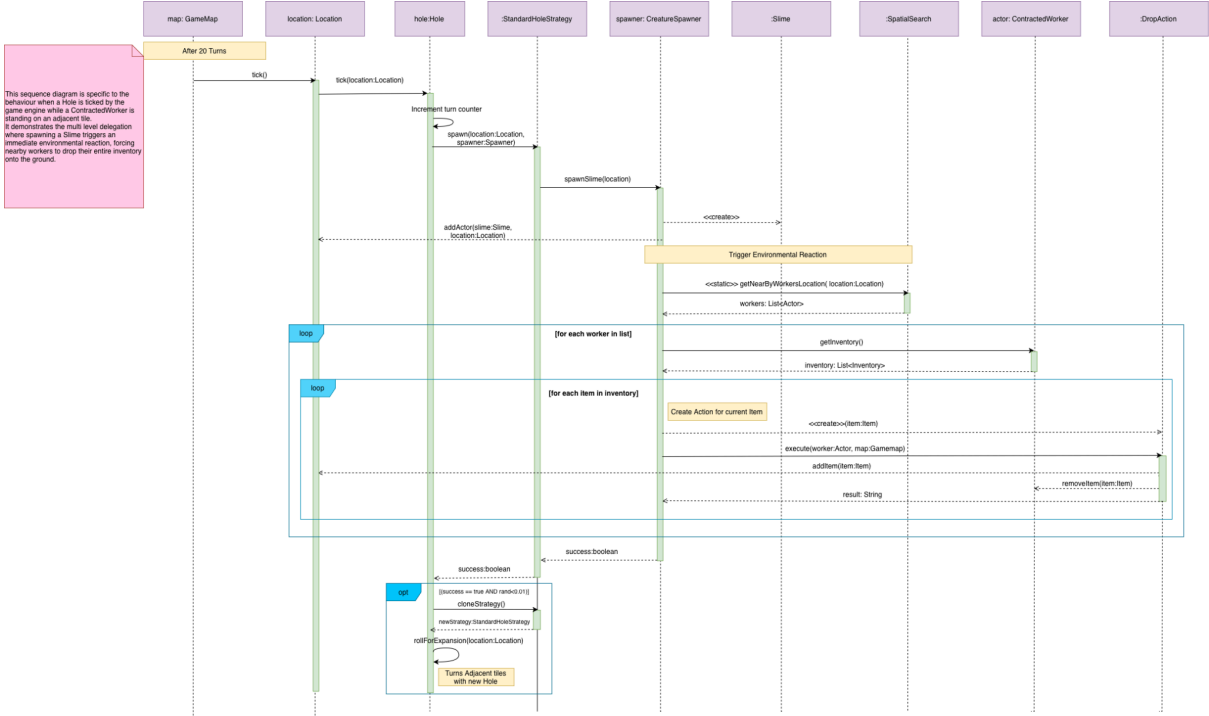
Finally, the execute() methods within the stage classes are designed to strictly enforce the single-threaded constraint. By utilizing a priority based branching structure where spawning triggers an immediate return, the system ensures that a tree cannot perform two major actions in a single turn. The separation of the updateAge() internal state update from the primary action ensures that the tree's growth clock is consistent and fair, demonstrating a commitment to the KISS principle by keeping the control flow deterministic and easy to debug.

REQ4: New Creature Detected

REQ 4 UML Class Diagram



REQ 4 Sequence Diagram for Hole spawn behaviour with worker drop reaction.



REQ 4 Design Rationale

A. Implementation of the Parasite Actor and Behavior Priority

Design 1: Override playTurn() in the Parasite class to hardcode the check for nearby hosts before moving

Pros	Cons
Simple and requires only one class file for the Parasite.	Violates OCP: Adding a third priority (like "Run away from Worker") would require modifying the Parasite's core logic.
Logic is localized to the actor.	Violates DRY: The wandering logic would likely be duplicated from other NPC classes.

Design 2 (Chosen): Extend abstract NonPlayerCharacter and use a TreeMap-based Behavior Priority System

The Parasite extends NonPlayerCharacter, which provides a TreeMap<Integer, Behaviour> to manage actions. In its constructor, the Parasite registers InfectBehaviour at priority 1 and WanderBehaviour at priority 999.

Pros	Cons
OCP is satisfied: The Parasite is "closed" for modification but "open" for extension; we can change its personality by simply adding or removing behaviors in the constructor.	Introduces an extra layer of abstraction with the NonPlayerCharacter base class.
Separation of Concerns: The Parasite class is reduced to a simple "configuration" class that only defines its stats and behaviors, leaving the execution logic to the Behaviors.	Requires understanding the priority sorting logic of the TreeMap to know which action will fire first.
Reliable Priority: Using a TreeMap ensures the Parasite always checks for a host to infect (Priority 1) before it even considers moving (Priority 999).	

Final Design Justification

Design 2 was chosen to maintain consistency with the existing NPC architecture (Undead, Slime). By inheriting from NonPlayerCharacter, the Parasite benefits from a standardized turn-taking mechanism that iterates through behaviors in priority order. This cleanly implements the requirement that "if there is something it can infect, it will immediately infect it," as the InfectBehaviour is assigned the highest priority, ensuring movement only occurs if no valid host is found.

B. Implementation of the Parasite Infection Logic

Design 1: Implement infection logic directly inside a new CreatureAttackBehaviour that checks target types with instanceof or switch

Pros	Cons
Fewer classes needed. All infection routing logic lives in a single behaviour class.	Directly couples the behaviour to every infectable type (ContractedWorker, Undead, Lantern, Cookies). Adding a new host requires editing the behaviour class, an OCP violation.
The branching structure makes the different infection outcomes immediately visible to a reader in one place.	Using instanceof or switch on is a unit constraint and is a known code smell that signals the wrong abstraction.

	The behaviour class takes on responsibility for both target detection and host-specific reaction logic, violating SRP.
--	--

Design 2 (Chosen): New Infectable interface with reactToInfection() and updateInfection(); InfectBehaviour uses asCapability(Infectable.class)

The Infectable interface was newly created with two default methods: reactToInfection(Location) for the immediate moment of infection, and updateInfection(Location) for recurring per-turn effects. ContractedWorker, Undead, Lantern, and Cookies each implement this interface and define their own reactions. InfectBehaviour scans adjacent exits for actors using asCapability(Infectable.class) and falls back to location.getItemsAs(Infectable.class) for infectable items. If a host is found, it returns an InfectAction.

Pros	Cons
InfectBehaviour depends only on the Infectable interface, never on any concrete class. Adding a new host simply requires implementing Infectable, no changes to InfectBehaviour or InfectAction (OCP, DIP).	Requires creating the Infectable interface and having four classes implement it as a small upfront setup cost compared to the simpler Design 1.
asCapability(Infectable.class) uses the engine's built-in capability system, eliminating all instanceof checks in accordance with project constraints (DRY).	The default method bodies in the interface mean a class that implements Infectable but forgets to override a method will silently do nothing, which could be confusing during development.
Each host class owns its own reaction logic inside reactToInfection() and updateInfection(), satisfying SRP. The Parasite class itself is reduced to a constructor that registers InfectBehaviour and WanderBehaviour.	The item scanning path (getItemsAs) and actor scanning path (asCapability on actor) require two separate code branches in InfectBehaviour.
The two method interface cleanly separates the instant effect (reactToInfection) from the recurring effect (updateInfection), modelling the requirement precisely.	

Final Design Justification

Design 2 was chosen. The Infectable interface defines the contract: reactToInfection() fires once at the moment of contact and updateInfection() fires every turn while infected. InfectBehaviour calls asCapability(Infectable.class) on adjacent actors and getItemsAs(Infectable.class) for items on adjacent tiles, returning an InfectAction if a host is found. InfectAction calls target.reactToInfection() and then removes the Parasite from the map, as it dies immediately after infecting. This ensures InfectBehaviour and InfectAction

never import or reference any concrete host class (DIP). Each host's unique behaviour as the Worker becoming a living hive, the Undead exploding, the Lantern losing oil, and the Cookie pack depleting, is entirely encapsulated within that host's own `reactToInfection()` and `updateInfection()` implementations (SRP, OCP). Adding a new host type requires only implementing `Infectable`; no existing class changes.

C. Implementation of `InfectionStatus` (Ongoing Infection Effects)

Design 1: Poll `updateInfection()` manually in `ContractedWorker.playTurn()` and in each item's `tick()` method

Pros	Cons
Explicit as it is clear where the per-turn infection update is triggered by reading <code>playTurn()</code> or <code>tick()</code> .	<code>ContractedWorker.playTurn()</code> already handles consciousness checks, display, freeze detection, and disorientation. Adding infection polling is an SRP violation.
No new status class required; infection side effects are wired directly to the turn logic of the host.	Item <code>tick()</code> methods would each need bespoke infection-check logic. This duplicates the same pattern across Lantern and Cookies, violating DRY.
	If the engine's <code>tickStatuses()</code> is also called (as it is for health damage), the damage and update effects would need careful ordering to avoid double application.

Design 2 (Chosen): `InfectionStatus` extends `DamageOverTimeStatus`; `tickStatus()` delegates to host's `updateInfection()` via `asCapability(Infectable.class)`

`InfectionStatus` extends the existing `DamageOverTimeStatus` class, initialised with a duration of `Integer.MAX_VALUE` to model an indefinite infection. Its `tickStatus()` override first calls `super.tickStatus()` to apply 1 HP of damage per turn, then calls `entity.asCapability(Infectable.class)` to retrieve the host's `Infectable` implementation and triggers `updateInfection()`.

Pros	Cons
Reuses <code>DamageOverTimeStatus</code> for the health damage component, avoiding code duplication (DRY). The 1-HP per-turn damage is inherited, not reimplemented.	Extending <code>DamageOverTimeStatus</code> creates a deeper inheritance chain. A reader must understand the parent class to follow the full tick logic.

The engine calls tickStatuses() on every entity automatically each turn. InfectionStatus hooks into this existing mechanism without any modification to ContractedWorker.playTurn() or item tick() methods (KISS, SRP).	The indefinite duration via Integer.MAX_VALUE is a workaround rather than a purpose built concept, which may confuse a person reviewing the code unfamiliar with the intent.
asCapability(Infectable.class) keeps the status decoupled from all concrete host types as it works identically for actors and items (DIP, OCP).	If the engine ever changes how tickStatuses() is dispatched, the implicit contract that status ticking happens once per turn could silently break.
Using Integer.MAX_VALUE for duration cleanly models an indefinite infection without adding a special flag or override.	

Final Design Justification

Design 2 was chosen. DamageOverTimeStatus already provides the health tick structure; InfectionStatus simply extends it and adds the host specific updateInfection() call in its tickStatus() override. The engine's GameMap.tick() dispatches tickStatuses() to every entity each turn, so the infection updates automatically without any polling code in playTurn() or item tick() methods (DRY, KISS). The asCapability(Infectable.class) call inside tickStatus() means InfectionStatus never imports any concrete host class (DIP). The same InfectionStatus instance is used whether the host is a ContractedWorker, a Lantern, or a Cookies pack demonstrates that the design is open for extension without modification (OCP).

D. Implementation of the Parasitic Hole Strategy (20-overflow Moon)

Design 1: Add an if-check inside Hole.tick() to detect which map the hole is on, and branch to spawn either Undead/Slime or Undead/Parasite

Pros	Cons
No new class hierarchy needed. The map detection and spawn logic live entirely in Hole.tick().	Hole.tick() becomes responsible for determining its map context and selecting which creature spawn. The two additional responsibilities on top of managing the spawn timer, violating SRP.
The branching makes it straightforward to see which moon produces which creatures by reading a single method.	Adding a third moon with a different spawn pool requires modifying Hole.tick() again as an OCP violation.
	Map identification logic (comparing map names, coordinates, or references) introduces fragile coupling between the

	Hole and the specific EclipseNebula map setup.
--	--

Design 2 (Chosen): HoleSpawnStrategy interface with spawn() and cloneStrategy(); StandardHoleStrategy and ParasiticHoleStrategy as concrete implementations

A HoleSpawnStrategy interface was newly created with two methods: spawn(Location, Spawner) for executing the spawn logic and cloneStrategy() for duplicating the strategy when a hole expands. StandardHoleStrategy spawns Undead or Slime (50/50) for the 99-deprecated moon. ParasiticHoleStrategy spawns Undead or Parasite (50/50) for the 20-overflow moon. The Hole constructor accepts a HoleSpawnStrategy instance, and EclipseNebula wires the correct strategy to each hole at construction time.

Pros	Cons
Hole.tick() has a single responsibility: managing the 20-turn counter and delegating the spawn decision to the injected strategy (SRP, OCP).	Requires creating the HoleSpawnStrategy interface and two concrete strategy classes making more code than the simple if-chain.
Adding a third moon with a unique creature pool requires only implementing a new HoleSpawnStrategy as Hole itself never changes (OCP, DIP).	EclipseNebula must know which strategy to inject for each hole, meaning the moon to strategy mapping still lives somewhere, just moved to the composition root.
cloneStrategy() supports the hole expansion mechanic cleanly: the newly created adjacent hole inherits an identical strategy instance without Hole needing to know the concrete type (OCP, DIP).	cloneStrategy() relies on convention. If a new strategy is to be implemented it must remember to return a fresh instance, not this.
Dependency injection of the strategy via the constructor follows the Dependency Inversion Principle as Hole depends on the HoleSpawnStrategy abstraction, not on any concrete creature or map class.	

Final Design Justification

Design 2 was chosen. The Strategy Pattern isolates the creature-pool decision from the Hole's lifecycle management. Hole.tick() manages the 20-turn counter and calls strategy.spawn(location, spawner) as it never knows or cares whether it is on the 99-deprecated or 20-overflow moon. When the 1% hole expansion check triggers, Hole calls strategy.cloneStrategy() and passes the result to the new adjacent Hole, guaranteeing identical spawning behaviour without any knowledge of the concrete strategy type (DIP). EclipseNebula acts as the composition root and wires the correct strategy at construction time, keeping the mapping in one well documented place. This satisfies OCP cleanly: new moons and new creature pools are introduced via new HoleSpawnStrategy implementations with zero changes to Hole itself.

E. Implementation of the Vent Spawner (20-overflow Moon)

Design 1: Add Vent logic directly to the existing Hole class with a boolean `isMotionActivated` flag

Pros	Cons
Reuses the Hole class without introducing a new ground type.	Hole and Vent have fundamentally different activation conditions: Holes tick unconditionally every 20 turns, while Vents only activate when a worker is adjacent. Merging them violates SRP.
The display character and spawn timer logic already exist in Hole, reducing duplication.	The boolean flag approach requires branching inside <code>tick()</code> to decide the activation condition, making the class harder to extend and understand.
	Vent's poison has a side effect on all adjacent actors, including the newly spawned creature as it is unrelated to Hole's existing responsibilities and would further bloat the class.

Design 2 (Chosen): Separate Vent class extending Ground; motion-activated `tick()` checking for adjacent `IS_WORKER` actors; poison applied via `PoisonStatus`

Vent is a separate Ground subclass. Its `tick()` method checks adjacent exits for any actor with the `WORKER` ability using `hasAbility(Ability.WORKER)`. If a worker is detected, the Vent attempts a spawn via its injected `HoleSpawnStrategy`. On a successful spawn, all actors in adjacent tiles, including the new creature, are given a `PoisonStatus` for 5 turns.

Pros	Cons
Vent and Hole have completely separate classes with clearly distinct responsibilities: Hole ticks on a fixed interval, Vent ticks on motion detection. SRP is fully satisfied.	Introduces an additional Ground subclass. Vent shares some setup code with Hole (display character, strategy injection) that cannot be fully refactored without a common abstract parent.
Vent reuses the <code>HoleSpawnStrategy</code> interface for its creature pool, meaning <code>ParasiticHoleStrategy</code> works for both Holes and Vents on the 20-overflow moon without modification (DRY, OCP).	The motion-activation check runs every turn for every Vent tile even when no workers are near, adding minor overhead on large maps.

Worker detection uses <code>hasAbility(Ability.WORKER)</code> rather than <code>instanceof</code> , maintaining the project's no- <code>instanceof</code> constraint (DIP).	Poisoning the newly spawned creature requires locating it after spawn, which couples the poison application to the spawner's placement logic.
The poison side effect is applied as a <code>PoisonStatus</code> added to adjacent actors, reusing the existing status infrastructure without modifying the spawn or turn logic (OCP, DRY).	

Final Design Justification

Design 2 was chosen. Vent's activation condition (worker adjacency) is semantically and structurally different from Hole's fixed-interval condition. Combining them into one class would require a branching flag that grows with each new spawner variant as an OCP violation. A separate Vent class keeps each ground type focused on a single spawn trigger (SRP). The `HoleSpawnStrategy` abstraction is reused unchanged, demonstrating that the interface is genuinely open for extension: `ParasiticHoleStrategy` was designed for Holes but works equally for Vents on the same moon (OCP, DIP). Worker detection via `hasAbility(Ability.WORKER)` avoids `instanceof` and allows any future worker type to trigger Vents automatically. The poison side effect is applied by iterating exits after a successful spawn and adding `PoisonStatus` to all adjacent actors, cleanly separating the spawn concern from the environmental hazard concern.

F. Hole Expansion Mechanic (1% Chance on Successful Spawn)

Design 1: Hole stores a reference to its own type and uses reflection or a factory method to instantiate a new Hole of the same type on an adjacent tile

Pros	Cons
No <code>cloneStrategy()</code> method needed on <code>HoleSpawnStrategy</code> then the Hole recreates itself via a known constructor signature.	Using reflection to instantiate a Hole is fragile, verbose, and introduces a runtime dependency on the constructor signature remaining stable.
Keeps the expansion logic self-contained within the Hole class.	A factory method that returns <code>new Hole(...)</code> couples the Hole to its own concrete constructor arguments, creating a hidden dependency on the exact configuration that was used at setup time.
	Either approach makes it difficult to share the same expansion logic with Vent without duplicating the mechanism.

Design 2 (Chosen): cloneStrategy() on HoleSpawnStrategy; Hole creates a new Hole with strategy.cloneStrategy() on a random adjacent IS_FLOOR tile

Pros	Cons
cloneStrategy() is a lightweight Prototype pattern: each strategy implementation returns a fresh copy of itself. No reflection, no factory, no coupling to constructor arguments (KISS).	Developers adding new HoleSpawnStrategy implementations must remember to implement cloneStrategy() correctly. Returning this instead of a new instance would silently share state between parent and child holes.
Hole never needs to know its own concrete type or how it was built. It delegates the strategy duplication entirely to the strategy object (DIP, SRP).	The 1% expansion probability means the mechanic is rare and difficult to test without mocking Random.
The expansion mechanic works identically for any HoleSpawnStrategy implementation, including future ones, without modifying Hole (OCP).	
Finding a random adjacent IS_FLOOR tile uses hasAbility(Ability.IS_FLOOR), consistent with the project's no-instanceof constraint (DIP).	

Final Design Justification

Design 2 was chosen. The cloneStrategy() method on HoleSpawnStrategy follows the Prototype Pattern: each concrete strategy knows how to copy itself. When the 1% expansion check triggers in Hole.tick(), the code calls strategy.cloneStrategy() and passes the result to a new Hole placed on a random adjacent IS_FLOOR tile. Hole never references any concrete strategy class, and it does not need to know how it was originally configured (DIP, SRP). StandardHoleStrategy.cloneStrategy() returns new StandardHoleStrategy() and ParasiticHoleStrategy.cloneStrategy() returns new ParasiticHoleStrategy() and has two lines of code each, no reflection, no factory overhead (KISS). Future strategy implementations automatically inherit the expansion mechanic by implementing cloneStrategy(), with no changes to Hole required (OCP).

G. Spawning Side Effects via the CreatureSpawner (Spawner Interface)

Design 1: Apply spawning side effects directly inside each spawner's ground class (Hole.tick(), Vent.tick())

Pros	Cons
------	------

Side-effect logic is co-located with the spawn trigger, making it easy to trace the cause and effect in a single class.	The same side effects (Undead health boost, Slime item drop, Parasite damage) must be duplicated in every ground class that can spawn creatures. Hole (two variants), Vent, and any infected entity. This is a direct DRY violation.
No additional abstraction layer needed as ground classes handle both activation and consequence.	Ground classes would take on responsibility for actor manipulation (hurting workers, dropping items), violating SRP. Ground subclasses should manage tile state, not creature lifecycle.
	Testing the side effects would require instantiating a full Ground object and a GameMap, increasing test complexity.

Design 2 (Chosen): Spawner interface with spawnUndead(), spawnSlime(), spawnParasite(); CreatureSpawner implements all side effects centrally; injected into Hole, Vent, and infected hosts

The Spawner interface declares spawnUndead(Location), spawnSlime(Location), and spawnParasite(Location). CreatureSpawner implements all three, encapsulating the side effects: spawnUndead() boosts Max HP by the count of nearby non-worker creatures; spawnSlime() forces adjacent workers to drop their inventory via DropAction; spawnParasite() deals 2 damage to adjacent workers. The Spawner is injected into Hole, Vent, ContractedWorker (for parasite spawning when infected), and Cookies via constructor injection.

Pros	Cons
All three spawning side effects are implemented exactly once in CreatureSpawner. Any ground or entity that needs to spawn a creature simply calls the appropriate Spawner method. There is no duplication (DRY).	Constructor injection of Spawner into ContractedWorker, Cookies, and the ground classes adds parameters to multiple constructors, which EclipseNebula must wire correctly as the composition root.
Ground classes and infected hosts depend on the Spawner interface, not on CreatureSpawner. This means different spawner implementations can be swapped in for testing or future expansion without modifying any caller (DIP, OCP).	CreatureSpawner grows as new creature types with new side effects are added. If it becomes too large, it may need to be split by creature type.

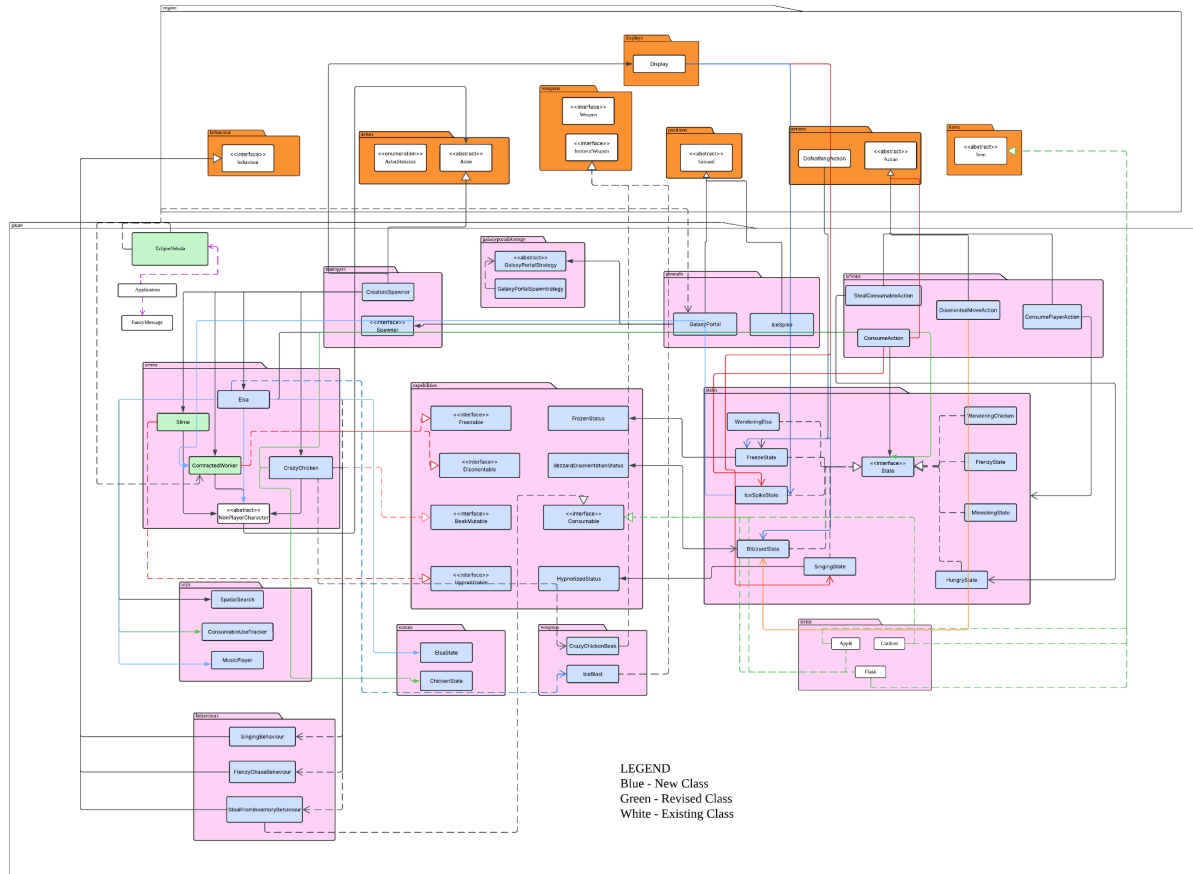
SRP is satisfied at every level: Hole manages the timer and delegates spawning; Vent manages motion detection and delegates spawning; CreatureSpawner manages creature creation and side effects.	
SpatialSearch is used inside CreatureSpawner to find nearby workers and actors, centralising all spatial queries in a single utility class (DRY).	

Final Design Justification

Design 2 was chosen. The Spawner interface creates a single point of responsibility for all creature creation and environmental reactions. Without it, the Parasite spawn damage (2 HP to adjacent workers) would need to be copied into Hole.tick(), Vent.tick(), ContractedWorker's infection update, and Cookies' infection update together four copies of identical logic. CreatureSpawner writes this once and all callers invoke spawner.spawnParasite(location). The Spawner interface also satisfies DIP: the infected ContractedWorker does not import CreatureSpawner; it calls the injected Spawner abstraction. EclipseNebula acts as the composition root, constructing a single CreatureSpawner instance and injecting it everywhere it is needed. This ensures that adding a fourth spawning side effect requires only one change, inside CreatureSpawner, with no modifications to any caller (OCP).

REQ5: Stateful creatures

REQ5 UML DIAGRAM



REQ5 SEQUENCE DIAGRAM

The sequence diagram shows the scenario when Elsa enters the SINGING state, she prints three messages, plays 'Let It Go' music, then iterates through every map tile to find slimes. Each slime receives HypnotizedStatus for 5 turns. While singing, Elsa wanders aimlessly. The state ends after 5 turns or when no slime remains adjacent. The state ends after 5 turns or when no slime remains adjacent.



REQ5: Stateful creatures Design Rationale

A. State Management Architecture

Design Decision: Generic State Pattern with Enum-Based Transitions

The design utilizes a generic `State<T>` extends `Enum<T>>` interface with concrete state classes for each behavior, coupled with `EnumMaps` for state storage.

Design 1: Direct Conditional Logic (if-else/switch inside Actor)

Pros	Cons
Fewer files needed. Only <code>Elsa.java</code> and <code>CrazyChicken.java</code> .	Violation of OCP. Adding a 6th state like "MELTING" requires modifying <code>Elsa.playTurn()</code> and adding another else-if block and modifying existing transition conditions. This risks breaking existing states.
Simple to implement initially. Able to write all 5 states in one file in a few hours without creating 9 separate state classes	Violation of SRP. Elsa becomes responsible for: wandering logic, freeze logic, blizzard logic, ice spike logic, singing logic + transitions + effects + actions = ~300 lines in ONE class
Easier debugging, as it has a single flow to trace, we can put a breakpoint in <code>playTurn()</code> and see everything in one method	The Elsa class would contain EVERYTHING about all 5 states. Changing freeze duration affects the same file as singing music. Merge conflicts become very difficult.
	Cannot test <code>SingingState</code> in isolation - must instantiate full Elsa actor with all dependencies
	There would be many duplicated codes. Each state's wander action would <code>repeatwanderBehaviour.operate()</code> 5 times.

Design 2: State Pattern with Generic Interface (Current Design)

Pros	Cons
Adheres to SRP. SingingState ONLY handles singing logic. FreezeState ONLY handles freezing. FrenzyState ONLY handles frenzy. Each class has ONE reason to change.	Takes up memory. Each Elsa holds an EnumMap with 5 state references. Each state instance is shared across all Elsa instances
Deterministic transitions, where each state's getNextState() is self-contained.	Increased file count. Developers must open multiple files to understand the full state machine.
Liskov Substitution, any State<ElsaState> can be swapped. The Elsa class doesn't care which state it's in since it just calls getAction(), getNextState(), onEnter(), onExit()	
Testable in isolation	
Reusable pattern. CrazyChicken uses the EXACT same State<T> interface with ChickenState enum. No duplicate architecture.	

Justification: Design 2 was chosen to satisfy SRP and OCP. The requirement needs deterministic transitions and at least 4 states per creature. The State Pattern encapsulates each state's unique behavior (transitions, actions, entry/exit effects) into separate classes. This eliminates if-else chains that would violate OCP. The generic State<T> extends Enum<T>> interface allows both ElsaState and ChickenState enums to be used polymorphically. The EnumMap provides O(1) state lookup while maintaining type safety.

B . Deterministic State Transitions

Design Decision: Complex Condition-Based Transitions

The design uses deterministic conditions (distance checks, health thresholds, turn counters) rather than randomness or linear progression.

Design 1: Random or Modulo-Based

Pros	Cons
Easy to implement. Only one line of code: <code>return ElsaState.values()[random.nextInt(5)];</code> - one line of code.	Cannot guarantee behaviour. Same conditions produce different results. A worker at 1 HP might trigger ICE_SPIKE one time and WANDERING another time.
Creates unpredictability	Violates "complex logic" rule. Random selection is trivial, not complex. No distance checks, no health thresholds, no worker counts.
	The Elsa class would contain EVERYTHING about all 5 states. Changing freeze duration affects the same file as singing music. Merge conflicts become very difficult.
	Goes against rubric.

Design 2: Linear Chain (A→B→C→D)

Pros	Cons
Easy to predict. WANDERING → FREEZE → BLIZZARD → ICE_SPIKE → SINGING → WANDERING	No branching logic. Cannot have different behavior based on environment. Elsa would always freeze, then blizzard, then ice spike, regardless of worker positions or health.
	Does not meet 2+ states requirement. Each state transitions to exactly 1 other state (the next in chain). Requirement requires transitions to ≥ 2 states.

Justification: Design 2 was chosen to satisfy SRP and OCP. The requirement needs deterministic transitions and at least 4 states per creature. The State Pattern encapsulates each state's unique behavior (transitions, actions, entry/exit effects) into separate classes.

This eliminates if-else chains that would violate OCP. The generic `State<T>` extends `Enum<T>>` interface allows both `ElsaState` and `ChickenState` enums to be used polymorphically. The `EnumMap` provides $O(1)$ state lookup while maintaining type safety.

Design 3: Complex Deterministic Conditions

Pros	Cons
Uses complex logic, like distance checks, worker counts, and health thresholds.	More complex to implement. <code>WanderingElsa.getNextState()</code> requires calling <code>SpatialSearch</code> methods and checking multiple conditions in correct priority order. Wrong order could cause unintended transitions.
Fully deterministic. Same conditions always produce same result. If a worker is at 2/10 HP (20%), <code>hasLowHealthWorker</code> returns true every time, so Elsa always transitions to <code>ICE_SPIKE</code> . No randomness.	

Justification: Design 3 was strictly required by HD specifications. The transitions must be deterministic (no `Random`), must involve complex logic (distance calculations, health thresholds, inventory inspection), and each state must transition to at least 2 other states. Elsa states transition to 4 others; Chicken states transition to 2-3 others. The implementation uses `SpatialSearch` utility methods to compute distances and counts, ensuring consistency and testability.

C . Immediate State Entry Effects

The design implements complex environmental effects upon state entry rather than simple healing/hurting.

Design 1: Simple Effects (Healing/Damage Only)

Pros	Cons
Easy to implement, <code>target.hurt(5);</code> or <code>actor.heal(3);</code> uses one line of code per effect.	Uninteresting gameplay. Players don't see any meaningful environmental change. Just numbers decreasing.
Low bug risk as simple integer operations rarely fail.	No iteration over map, no terrain modification, no status effects, no audio, no item manipulation.
	Forbidden in rubric, does not meet HD requirement

Design 2: Complex Environmental Effects

Pros	Cons
Meets the requirement of having complex logic. For example, <code>FreezeState.onEnter()</code> iterates entire map: <pre>for (int y : map.getYRange()) { for (int x : map.getXRange()) { ... } }</pre> - applies <code>FrozenStatus</code> to every worker.	Potential for unexpected interactions.

Justification: Design 2 was required by HD specifications. Simple effects like "heal 5 HP" or "deal 2 damage" are explicitly stated as insufficient. Each state entry effect must involve "sufficiently complex logic." The implementation provides: global map iteration (FREEZE, BLIZZARD, SINGING), terrain generation (ICE_SPIKE), physics-based pushback with pathfinding (FRENZY), and item pulling with radius search (HUNGRY).

D . Capability Pattern

The design uses capability interfaces and `asCapability()` method instead of direct instanceof checking.

Design 1: Direct instanceof Checking

Pros	Cons
Simple syntax and faster execution.	In unit tests, you cannot create a mock object that passes instanceof Freezable without using complex mocking frameworks (PowerMock).
	Tight coupling, FreezeState would directly reference ContractedWorker class. If the worker class is renamed, the state breaks.

Design 2: Capability Pattern with `asCapability()`

Pros	Cons
Adheres to DIP. FreezeState.onEnter() calls target.asCapability(Freezable.class).ifPresent(f -> f.freeze(duration)). The state depends on the Freezable interface, not on ContractedWorker class.	Requires each class to implement capability methods. Every class that can be frozen must implement Freezable.freeze(). ContractedWorker implements it. If a new SecurityGuard actor is added, it must also implement Freezable.freeze().
Interface Segregation. Slime implements Hypnotizable but NOT Freezable. CrazyChicken implements BeakMutable but NOT Hypnotizable. Each actor implements ONLY the capabilities it needs	

Justification: The capability pattern was chosen over instanceof to satisfy the Open-Closed Principle and Dependency Inversion Principle. As demonstrated in REQ5, status checking uses class comparison (`status.getClass() == HypnotizedStatus.class`) rather than instanceof to avoid false positives from inheritance. This ensures that only the exact status type triggers the behavior, following Ed discussion requirements.

E . Status Effects for Workers

The design uses engine-integrated Status objects with automatic duration tracking rather than simple boolean flags.

Design 1: Simple Boolean Flags

Pros	Cons
Extremely simple. private boolean frozen = true; in ContractedWorker.	Cannot track duration. Worker would be frozen forever unless manually unfrozen. No automatic expiration after 2 turns.
No additional classes.	No integration with engine tick. Would need to manually decrement counters in ContractedWorker.playTurn(), adding complexity to an already complex method.
	Not reusable. Each actor would need its own boolean flags. Cannot share status logic.

Design 2: Status Pattern with Duration

Pros	Cons
Automatic duration tracking. FrozenStatus.tickStatus() decrements remainingTurns each turn. When remainingTurns == 0, isActive() returns false. No manual reset needed.	More classes to maintain.
Engine integration. The engine calls tickStatus() on every Status attached to every actor each turn. Statuses manage themselves.	More memory needed.
Class comparison for checking. ContractedWorker.isFrozen() uses status.getClass() == FrozenStatus.class - NOT instanceof. This ensures only exact FrozenStatus matches, not subclasses.	

Justification: The Status Pattern was chosen over simple flags because REQ5 requires duration-based effects (2 turns frozen, 3 turns disoriented). The engine's Status interface provides built-in tickStatus() for automatic duration management. The freeze check uses class comparison (==) rather than instanceof as required by Ed discussion to ensure exact type matching without inheritance false positives. The disorientation effect wraps movement actions rather than modifying the actor directly, preserving the original action structure while randomizing movement.

F. Spatial Search Utility

Design 1: Each State Implements Own Detection

The design uses a static utility class `SpatialSearch` to centralize all distance and actor detection logic.

Pros	Cons
No extra utility class. Each state manages its own loops.	Code duplication. All character's states would each need to implement 8-way exit traversal. That's 9 copies of the same 10-line loop.
	Inconsistent behavior. One state might use Manhattan distance, another Euclidean distance. <code>FrenzyState</code> might have different detection radius than <code>MimickingState</code> due to copy-paste errors.

Design 2: Centralized `SpatialSearch`

Pros	Cons
Eliminates code duplication. All 9 state classes call <code>SpatialSearch.hasWorkerWithinDistance()</code> instead of writing their own loops. Single implementation of distance calculation.	Static coupling. All 9 state classes directly call <code>SpatialSearch.hasWorkerWithinDistance()</code> . Cannot easily swap to a different search algorithm without modifying all callers.
Single source of truth, <code>SpatialSearch.calculateDistance()</code> uses Manhattan distance: <code>Math.abs(a.x() - b.x()) + Math.abs(a.y() - b.y())</code> . Every state uses the EXACT same calculation.	All logic in one class. <code>SpatialSearch.java</code> contains 200+ lines of detection methods. Could become a "God Utility" class over time.
Used across multiple REQs. REQ3 flora uses <code>SpatialSearch.getNearbyWorkers()</code> . REQ4 spawners use <code>SpatialSearch.getNearbyActors()</code> . REQ5 states use all methods. Consistency across entire codebase.	

Justification: Centralizing detection logic in `SpatialSearch` follows the DRY Principle and prevents Shotgun Surgery. Without this utility, each of the 9 state classes (and `CreatureSpawner`) would need to implement the same 8-way exit traversal and worker capability checking. The static coupling trade-off is acceptable as this is infrastructure-level code with no side effects, and the methods are pure functions with deterministic outputs.

G. Galaxy Portal Strategy Pattern

The design uses the Strategy Pattern for Galaxy Portal spawning, following the pattern established in REQ4.

Design 1: Hardcoded Spawn Logic

Pros	Cons
Simple implementation.	Violation of OCP. To add new portal that spawns 100% CrazyChickens, we would need to modify GalaxyPortal.java to either add a boolean flag or create a subclass. Modifying existing code risks breaking normal behavior.
	Cannot reuse portal. Each portal instance would have the same spawn behavior. Cannot have different portals with different spawn rules on the same map.
	Difficult to test.

Design 2: Strategy Pattern

Pros	Cons
Adheres to OCP by adding a new spawn strategy (e.g., ElsaOnlyStrategy) requires creating a new class and doesn't modify GalaxyPortal.	Additional classes. For each new strategy, another class file.
Reusable portal. Different portal instances can have different strategies: new GalaxyPortal(new GalaxyPortalStrategy()) for 50/50, new GalaxyPortal(new ElsaOnlyStrategy()) for testing.	
Follows existing pattern. REQ4 uses HoleSpawnStrategy with StandardHoleStrategy and ParasiticHoleStrategy. REQ5 GalaxyPortalStrategy follows exact same pattern, maintaining consistency.	

Justification: The Strategy Pattern was chosen to follow the same design pattern established in REQ4 (which used HoleSpawnStrategy with StandardHoleStrategy and ParasiticHoleStrategy). This maintains consistency across the codebase and adheres to the Open-Closed Principle. New spawn behaviors (e.g., a "Halloween" portal that spawns 100% CrazyChickens, or a "Testing" portal that spawns only Elsa) can be added by creating a new strategy class without modifying GalaxyPortal. The pattern also enables dependency injection, making the portal testable with mock strategies.

H . Movement Disorientation

Design Decision: Disoriented Movement with 4-Direction Limitation

When Elsa enters the BLIZZARD state, all workers on the map receive BlizzardDisorientationStatus for 3 turns. While disoriented, workers only see 4 movement options (North, South, East, West) instead of the usual 8 directions, and their chosen movement direction is mirrored. North goes South and East goes West.

Design 1: 8-Direction disorientation

Pros	Cons
All 8 directions remain available	Tried to implement but kept getting bugs. Difficult to implement.
Maximum player choice	

Design 2: 4-Direction Only with Direction Wrapping

Pros	Cons
Preserves player agency - player still chooses which direction to attempt	Requires modifying the action list before displaying menu
Creates genuine disorientation - player cannot guarantee where they will move	Less complex than 8-direction choices.
Eliminates diagonals - reduces choices from 8 to 4, making navigation intentionally harder	
Uses class comparison to check for <code>BlizzardDisorientationStatus</code>	

Justification: The HD requirement explicitly states that at least two out of four states must involve creating one new behaviour that has not existed in the implementation so far. For a group of 5, this means 4 out of 8 states require new behaviors. The implementation exceeds this with 6 new behaviors across both creatures. Each new behavior involves complex logic beyond simple healing or hurting, as demonstrated by the complexity descriptions above. The behaviors cannot be achieved by simply combining existing engine behaviors like WanderBehaviour and AttackBehaviour. The disorientation effect specifically provides a new behavior through movement wrapping and 4-direction limitation, uses class comparison with `status.getClass() ==` for status checking rather than `instanceof`, applies an immediate entry effect when Elsa enters BLIZZARD state, and lasts for a precise duration of 3 turns while the decision to enter BLIZZARD uses worker counts and is fully deterministic.

Summary

The implementation of Elsa (5 states) and CrazyChicken (4 states) successfully satisfies all HD requirements for REQ5. The design demonstrates a robust application of the State Pattern, Capability Pattern, Strategy Pattern, and SOLID principles to create deterministic, stateful creatures with complex transition logic and immediate entry effects.

Both creatures exhibit fully deterministic behavior. No randomness is used in state transitions. Each state is able to transition to at least 2-4 other states based on their complex conditions like distance checks, worker population within a map, worker's health percentages ($\leq 50\%$), worker inventory scanning for consumables, and duration tracking. Entry effects are sufficiently complex, featuring global map iteration (FREEZE, BLIZZARD, SINGING), terrain generation (ICE_SPIKE), physics-based pushback with pathfinding (FRENZY), and item pulling with radius search (HUNGRY).

The design introduces 6 novel behaviors that did not exist in previous requirements: global freeze application, movement action wrapping for disorientation, global hypnotization with music playback, shockwave damage with pushback, consumable item pulling, and mirror movement behavior. All code avoids instanceof by using the capability pattern (asCapability()) and class comparison (status.getClass() ==) for status checking.